

Beschreibungslogiken: Semantik, Schlussfolgern und Lernen

Eine formale Einführung mit vollständigen Beweisen

Stephan Epp

Institut für Informatik, Universität Bielefeld

8. Juni 2026

Zusammenfassung

Diese Arbeit gibt eine umfassende formale Einführung in Beschreibungslogiken (Description Logics, DL), die das logische Fundament des Semantic Web und des OWL-2-Standards bilden. Wir entwickeln die Semantik der wichtigsten DL-Familien (EL, ALC, SHIQ, SROIQ) von Grund auf, beweisen alle zentralen Ergebnisse vollständig und formal, darunter die Polynomialzeit-Komplexität der Subsumtion in EL, die PSPACE-Vollständigkeit von ALC, die Korrektheit des Tableau-Verfahrens sowie Existenz und Berechnung von LCS und MSC für das symbolische Konzeptlernen. Abgerundet wird die Darstellung durch ausführliche Analysen zu nicht-monotonem Schlussfolgern, relaxierter Anfragebeantwortung und einem weitreichenden Ausblick auf offene Forschungsfragen.

Inhaltsverzeichnis

1	Einführung und Motivation	3
1.1	Wissensrepräsentation als Forschungsgebiet	3
1.2	Was sind Beschreibungslogiken?	3
1.3	Historische Entwicklung	3
1.4	Kernbeiträge dieses Dokuments	3
2	Formale Grundlagen	4
2.1	Signaturen und syntaktische Grundbegriffe	4
2.2	Interpretationen und Semantik	5
2.3	TBox, ABox und Wissensbasen	7
3	Die Beschreibungslogik EL	9
3.1	Syntax und Semantik von EL	9
3.2	Normalisierung von EL-TBoxen	9
3.3	Der EL-Subsumtionsalgorithmus	10
4	Die Beschreibungslogik ALC	12
4.1	Ausdrucksstärke von ALC	12
4.2	Verbindung zur Modallogik	12
4.3	PSPACE-Vollständigkeit von ALC	13

5	Tableau-basierte Entscheidungsverfahren	14
5.1	Formale Definition des Tableau-Verfahrens	14
5.2	Korrektheit und Vollständigkeit des Tableau-Verfahrens	15
6	Konzepthierarchie, Subsumtion und Instanzprüfung	17
6.1	Formale Eigenschaften der Subsumtionsrelation	17
6.2	Instanzüberprüfung und Realisierung	18
7	Erweiterte DL-Familien: SHIQ und SROIQ	18
7.1	SHIQ: Transitive Rollen, Rollenhierarchien und Zählen	18
7.2	SROIQ und OWL 2	19
8	Lernen von Beschreibungslogik-Konzepten	19
8.1	Das Lernproblem	19
8.2	Most Specific Concept (MSC)	20
8.3	Least Common Subsumer (LCS)	20
9	Nicht-monotones Schlussfolgern	21
9.1	Motivierende Beispiele und Probleme	21
9.2	Defeasible Inklusionen	22
10	Relaxierte Anfrageauswertung	23
10.1	Konjunktive Anfragen	23
10.2	Repair-basierte Inkonsistenztoleranz	23
11	Beschreibungslogik und der Subgraph-Algorithmus	24
11.1	Motivation: Zwei Welten, eine Struktur	24
11.2	DL-Interpretationen als gerichtete Graphen	25
11.3	Signatur-Kodierung für ABox-Individuen	27
11.4	Tableau-Blocking als Rotationsäquivalenz	28
11.5	Konjunktive Anfragen als Subgraph-Matching	29
12	Zusammenfassung	31
12.1	Ergebnisse im Überblick	32
12.2	Inhaltliche Zusammenfassung der Kapitel	32
13	Ausblick	33
13.1	Subgraph-basiertes DL-Reasoning	33
13.2	Skalierbare Reasoning-Algorithmen	34
13.3	Neuro-symbolische Integration	34
13.4	Erweiterungen der Ausdrucksstärke	35
13.5	Lerntheorie und Konzeptlernen	35
13.6	Linked Data, SPARQL und Wissensgraphen	36
13.7	Verbindungen zur Theoretischen Informatik	36
13.8	Anwendungen und Standardisierung	37
13.9	Offene Theoretische Fragen	37
13.10	Abschließende Einordnung	38

1 Einführung und Motivation

1.1 Wissensrepräsentation als Forschungsgebiet

Ein fundamentales Ziel der Künstlichen Intelligenz ist es, Wissen maschinell nutzbar zu machen. Wissensrepräsentation (Knowledge Representation, KR) ist das Teilgebiet, das sich mit der Frage beschäftigt: *Wie kann Wissen so formalisiert werden, dass ein Computer damit schlussfolgern kann?*

Historisch wurden verschiedene Formalismen entwickelt: Produktionsregeln (z. B. MYCIN [16]), semantische Netze [34], Frames [29] und schließlich logikbasierte Ansätze [28]. Beschreibungslogiken vereinen die intuitive Struktur semantischer Netze mit der formalen Präzision der Prädikatenlogik und entscheidbaren Inferenzverfahren [8].

1.2 Was sind Beschreibungslogiken?

Definition 1.1 (Beschreibungslogik (informal)). Eine *Beschreibungslogik* ist eine Familie formaler Sprachen zur Beschreibung von:

- **Konzepten** (Klassen von Objekten), z. B. *Arzt*, *Patient*,
- **Rollen** (binäre Beziehungen zwischen Objekten), z. B. *behandelt*,
- **Individuen** (konkrete Objekte), z. B. *alice*,

zusammen mit einer wohldefinierten modelltheoretischen Semantik und entscheidbaren Inferenzverfahren.

Beschreibungslogiken sind *Fragmente der Prädikatenlogik erster Stufe*. Sie verzichten auf volle Ausdrucksstärke zugunsten von Entscheidbarkeit und praktischer Effizienz. Je nach Ausdrucksstärke unterscheidet man verschiedene DL-Familien, die sich in einem Spektrum zwischen dem einfachen EL (polynomielle Komplexität) und dem ausdrucksstarken SROIQ (NEXPTIME) bewegen.

1.3 Historische Entwicklung

Die Wurzeln der Beschreibungslogiken liegen in der Arbeit von BRACHMAN und LEVESQUE [14], die 1985 zeigten, dass semantische Netze implizit logische Schlussfolgerungen erfordern, die im schlimmsten Fall PSPACE-schwer sind. Das System KL-ONE [13] war das erste strukturierte Wissensrepräsentationssystem mit einer zumindest teilweise formalen Semantik.

In den späten 1980er- und 1990er-Jahren wurden umfangreiche Komplexitätsanalysen durchgeführt [19, 36]. Eine Schlüsselbeobachtung war, dass ALC äquivalent zur multimodalen Logik K_m ist [36], was eine Brücke zur Modallogiktheorie schlägt.

Mit dem Aufkommen des World Wide Web und dem Ziel des Semantic Web wurden DL zur Grundlage von Ontologiebeschreibungssprachen: OWL (basierend auf SHOIN(D)) [30] und OWL 2 (basierend auf SROIQ(D)) [31] sind W3C-Standards.

1.4 Kernbeiträge dieses Dokuments

Wir beweisen folgende zentrale Resultate vollständig und formal:

- (I) **Injektivität der Signaturberechnung und Eindeutigkeit kanonischer Normalformen** (Abschnitt 2).
- (II) **Korrektheit und Vollständigkeit der ALC-Semantik** mit vollständiger Induktionsbeweissführung (Abschnitt 4).
- (III) **Polynomialzeit-Subsumtion in EL** mit vollständiger Analyse des Normalisierungs- und Regelanwendungsalgorithmus (Abschnitt 3).
- (IV) **PSpace-Vollständigkeit von ALC** durch explizite Reduktionen (Abschnitt 4.3).
- (V) **Korrektheit des Tableau-Verfahrens** für ALC, formal in beiden Richtungen (Abschnitt 5).
- (VI) **Existenz und Berechenbarkeit von LCS in EL** mit vollständiger Produktkonstruktion (Abschnitt 8).
- (VII) **Korrektheit der repair-basierten Anfragebeantwortung** (Abschnitt 10).

2 Formale Grundlagen

2.1 Signaturen und syntaktische Grundbegriffe

Definition 2.1 (Signatur). Eine *Signatur* $\Sigma = (N_C, N_R, N_I)$ ist ein Tripel, bestehend aus einer abzählbaren Menge von *Konzeptnamen* N_C , einer abzählbaren Menge von *Rollennamen* N_R und einer abzählbaren Menge von *Individuennamen* N_I , wobei N_C , N_R und N_I paarweise disjunkt sind:

$$N_C \cap N_R = \emptyset, \quad N_C \cap N_I = \emptyset, \quad N_R \cap N_I = \emptyset.$$

Definition 2.2 (ALC-Konzeptausdrücke). Sei $\Sigma = (N_C, N_R, N_I)$ eine Signatur. Die Menge $\mathcal{C}_{\text{ALC}}(\Sigma)$ der *ALC-Konzeptausdrücke* über Σ ist die kleinste Menge $\mathcal{C}$, für die gilt:

- (i) $A \in \mathcal{C}$ für alle $A \in N_C$ (*Konzeptnamen*);
- (ii) $\top \in \mathcal{C}$ und $\perp \in \mathcal{C}$ (*universelles und leeres Konzept*);
- (iii) Wenn $C, D \in \mathcal{C}$, dann auch $\neg C \in \mathcal{C}$, $C \sqcap D \in \mathcal{C}$, $C \sqcup D \in \mathcal{C}$;
- (iv) Wenn $C \in \mathcal{C}$ und $r \in N_R$, dann auch $\exists r.C \in \mathcal{C}$ und $\forall r.C \in \mathcal{C}$.

Definition 2.3 (Größe eines Konzeptausdrucks). Die *Größe* $|C|$ eines Konzeptausdrucks C ist induktiv definiert:

$$\begin{array}{ll} |A| &= 1, & |\top| = |\perp| &= 1, \\ |\neg C| &= |C| + 1, & |C \sqcap D| = |C \sqcup D| &= |C| + |D| + 1, \\ |\exists r.C| &= |C| + 2, & |\forall r.C| &= |C| + 2. \end{array}$$

Definition 2.4 (Teilkonzepte). Die Menge $\text{sub}(C)$ der *Teilkonzepte* (sub-concepts) eines Konzeptausdrucks C ist induktiv definiert:

$$\begin{aligned}\text{sub}(A) &= \{A\}, \\ \text{sub}(\top) &= \{\top\}, \quad \text{sub}(\perp) = \{\perp\}, \\ \text{sub}(\neg D) &= \{\neg D\} \cup \text{sub}(D), \\ \text{sub}(C_1 \sqcap C_2) &= \{C_1 \sqcap C_2\} \cup \text{sub}(C_1) \cup \text{sub}(C_2), \\ \text{sub}(C_1 \sqcup C_2) &= \{C_1 \sqcup C_2\} \cup \text{sub}(C_1) \cup \text{sub}(C_2), \\ \text{sub}(\exists r.D) &= \{\exists r.D\} \cup \text{sub}(D), \\ \text{sub}(\forall r.D) &= \{\forall r.D\} \cup \text{sub}(D).\end{aligned}$$

Lemma 2.1 (Schranke für Teilkonzepte). Für jeden Konzeptausdruck C gilt $|\text{sub}(C)| \leq |C|$.

Beweis. Wir beweisen dies durch strukturelle Induktion über den Aufbau von C .

Basisfall: Für $C = A$, $C = \top$ oder $C = \perp$ gilt $\text{sub}(C) = \{C\}$, also $|\text{sub}(C)| = 1 = |C|$.
✓

Induktionsschritt $\neg D$: Nach Induktionshypothese gilt $|\text{sub}(D)| \leq |D|$. Dann:

$$|\text{sub}(\neg D)| = 1 + |\text{sub}(D)| \leq 1 + |D| = |\neg D|.$$

Induktionsschritt $C_1 \sqcap C_2$: Nach Induktionshypothese $|\text{sub}(C_i)| \leq |C_i|$ für $i = 1, 2$. Da $\text{sub}(C_1) \cap \text{sub}(C_2)$ möglicherweise nichtleer ist, gilt:

$$|\text{sub}(C_1 \sqcap C_2)| \leq 1 + |\text{sub}(C_1)| + |\text{sub}(C_2)| \leq 1 + |C_1| + |C_2| = |C_1 \sqcap C_2|.$$

Der Fall $C_1 \sqcup C_2$ ist analog.

Induktionsschritt $\exists r.D$:

$$|\text{sub}(\exists r.D)| = 1 + |\text{sub}(D)| \leq 1 + |D| < |D| + 2 = |\exists r.D|.$$

Der Fall $\forall r.D$ ist analog. □

2.2 Interpretationen und Semantik

Definition 2.5 (Interpretation). Eine *Interpretation* $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ besteht aus:

- einem nichtleeren *Interpretationsbereich* (Domain) $\Delta^{\mathcal{I}} \neq \emptyset$,
- einer *Interpretationsfunktion* $\cdot^{\mathcal{I}}$, die
 - jedem $A \in N_C$ eine Menge $A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$ zuordnet,
 - jedem $r \in N_R$ eine Relation $r^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$ zuordnet,
 - jedem $a \in N_I$ ein Element $a^{\mathcal{I}} \in \Delta^{\mathcal{I}}$ zuordnet.

Definition 2.6 (Semantik der ALC-Konstrukturen). Die Interpretationsfunktion wird auf komplexe Konzeptausdrücke durch folgende Gleichungen erweitert:

$$\begin{aligned}
\top^{\mathcal{I}} &= \Delta^{\mathcal{I}}, \\
\perp^{\mathcal{I}} &= \emptyset, \\
(\neg C)^{\mathcal{I}} &= \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}, \\
(C \sqcap D)^{\mathcal{I}} &= C^{\mathcal{I}} \cap D^{\mathcal{I}}, \\
(C \sqcup D)^{\mathcal{I}} &= C^{\mathcal{I}} \cup D^{\mathcal{I}}, \\
(\exists r.C)^{\mathcal{I}} &= \{d \in \Delta^{\mathcal{I}} \mid \exists e \in \Delta^{\mathcal{I}}. (d, e) \in r^{\mathcal{I}} \wedge e \in C^{\mathcal{I}}\}, \\
(\forall r.C)^{\mathcal{I}} &= \{d \in \Delta^{\mathcal{I}} \mid \forall e \in \Delta^{\mathcal{I}}. (d, e) \in r^{\mathcal{I}} \Rightarrow e \in C^{\mathcal{I}}\}.
\end{aligned}$$

Lemma 2.2 (Dualität von $\exists$ und $\forall$). Für jede Interpretation $\mathcal{I}$, jeden Rollennamen r und jeden Konzeptausdruck C gilt:

$$(\exists r.C)^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus (\forall r.\neg C)^{\mathcal{I}}.$$

Beweis. Wir zeigen Mengengleichheit durch gegenseitige Inklusion.

($\subseteq$): Sei $d \in (\exists r.C)^{\mathcal{I}}$. Dann existiert $e \in \Delta^{\mathcal{I}}$ mit $(d, e) \in r^{\mathcal{I}}$ und $e \in C^{\mathcal{I}}$, also $e \notin (\neg C)^{\mathcal{I}}$. Daher gilt $d \notin (\forall r.\neg C)^{\mathcal{I}}$ (da die Allaussage verletzt ist), also $d \in \Delta^{\mathcal{I}} \setminus (\forall r.\neg C)^{\mathcal{I}}$.

($\supseteq$): Sei $d \in \Delta^{\mathcal{I}} \setminus (\forall r.\neg C)^{\mathcal{I}}$. Dann gilt $d \notin (\forall r.\neg C)^{\mathcal{I}}$, d. h. es gibt $e \in \Delta^{\mathcal{I}}$ mit $(d, e) \in r^{\mathcal{I}}$ und $e \notin (\neg C)^{\mathcal{I}}$, also $e \in C^{\mathcal{I}}$. Damit gilt $d \in (\exists r.C)^{\mathcal{I}}$. $\square$

Korollar 2.1 (Pushdown von Negation). Für ALC-Konzepte gilt semantische Äquivalenz:

$$\begin{aligned}
\neg(C \sqcap D) &\equiv \neg C \sqcup \neg D, \\
\neg(C \sqcup D) &\equiv \neg C \sqcap \neg D, \\
\neg(\exists r.C) &\equiv \forall r.\neg C, \\
\neg(\forall r.C) &\equiv \exists r.\neg C.
\end{aligned}$$

Beweis. Alle vier Äquivalenzen folgen direkt aus der mengentheorie (De Morgan) und Lemma 2.2. Wir zeigen exemplarisch die dritte Gleichheit:

$$\begin{aligned}
(\neg(\exists r.C))^{\mathcal{I}} &= \Delta^{\mathcal{I}} \setminus (\exists r.C)^{\mathcal{I}} \\
&= \Delta^{\mathcal{I}} \setminus (\Delta^{\mathcal{I}} \setminus (\forall r.\neg C)^{\mathcal{I}}) \quad (\text{Lemma 2.2}) \\
&= (\forall r.\neg C)^{\mathcal{I}}.
\end{aligned}$$

$\square$

Definition 2.7 (Negationsnormalform). Ein ALC-Konzept C ist in *Negationsnormalform* (NNF), wenn alle Negationssymbole nur direkt vor Konzeptnamen stehen, also wenn C keine Teilkonzepte der Form $\neg(D_1 \sqcap D_2)$, $\neg(D_1 \sqcup D_2)$, $\neg\exists r.D$ oder $\neg\forall r.D$ enthält.

Satz 2.1 (NNF-Transformation). Jeder ALC-Konzeptausdruck C lässt sich in Polynomialzeit in eine äquivalente Negationsnormalform $\text{nnf}(C)$ transformieren, wobei $|\text{nnf}(C)| \leq 2|C|$.

Beweis. Wir definieren $\text{nnf}(C)$ induktiv und zeigen gleichzeitig (a) die Äquivalenz $C \equiv \text{nnf}(C)$ und (b) die Größenschranke $|\text{nnf}(C)| \leq 2|C|$.

Die Transformation wendet Korollar 2.1 erschöpfend an:

$$\begin{aligned}
\text{nnf}(A) &= A, \\
\text{nnf}(\top) &= \top, \quad \text{nnf}(\perp) = \perp, \\
\text{nnf}(\neg A) &= \neg A, \\
\text{nnf}(\neg\neg C) &= \text{nnf}(C), \\
\text{nnf}(\neg(C_1 \sqcap C_2)) &= \text{nnf}(\neg C_1) \sqcup \text{nnf}(\neg C_2), \\
\text{nnf}(\neg(C_1 \sqcup C_2)) &= \text{nnf}(\neg C_1) \sqcap \text{nnf}(\neg C_2), \\
\text{nnf}(\neg\exists r.C) &= \forall r.\text{nnf}(\neg C), \\
\text{nnf}(\neg\forall r.C) &= \exists r.\text{nnf}(\neg C), \\
\text{nnf}(C_1 \sqcap C_2) &= \text{nnf}(C_1) \sqcap \text{nnf}(C_2), \\
\text{nnf}(\exists r.C) &= \exists r.\text{nnf}(C), \\
\text{nnf}(\forall r.C) &= \forall r.\text{nnf}(C).
\end{aligned}$$

Äquivalenz (a): Jede Regel ersetzt einen Teilausdruck durch einen äquivalenten Ausdruck (Korollar 2.1, Doppelnegationnorm). Die Ersetzung erhält semantische Äquivalenz.

Größenschranke (b): Im schlimmsten Fall wird jede Negation durch doppelte Verwendung des NNF-Arguments der Teilkonzepte verdoppelt. Genauer gilt: Bei jedem Aufruf $\text{nnf}(\neg C)$ wird die Größe von C genau einmal gezählt, nie mehr als zweimal. Formal zeigt man $|\text{nnf}(\neg C)| \leq 2|C|$ und $|\text{nnf}(C)| \leq |C|$ durch gleichzeitige Induktion, was die Schranke $|\text{nnf}(C)| \leq 2|C|$ ergibt.

Laufzeit: Die Transformation bearbeitet jeden Knoten des Syntaxbaums in konstanter Zeit, der Baum hat $O(|C|)$ Knoten, also Gesamtlaufzeit $O(|C|)$. $\square$

2.3 TBox, ABox und Wissensbasen

Definition 2.8 (Allgemeine Konzeptinklusion (GCI)). Eine *allgemeine Konzeptinklusion* (General Concept Inclusion, GCI) ist ein Ausdruck der Form $C \sqsubseteq D$, wobei C und D ALC-Konzepte sind. Eine Interpretation $\mathcal{I}$ erfüllt $C \sqsubseteq D$ (geschrieben $\mathcal{I} \models C \sqsubseteq D$), wenn $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$.

Definition 2.9 (TBox). Eine *TBox* $\mathcal{T}$ ist eine endliche Menge von GCIs. Eine Interpretation $\mathcal{I}$ modelliert $\mathcal{T}$ (geschrieben $\mathcal{I} \models \mathcal{T}$), wenn $\mathcal{I} \models C \sqsubseteq D$ für alle $C \sqsubseteq D \in \mathcal{T}$.

Definition 2.10 (ABox-Assertionen und ABox). Eine *Konzeptassertion* ist ein Ausdruck $A(a)$ mit $A \in N_C$ und $a \in N_I$. Eine *Rollenassertion* ist ein Ausdruck $r(a, b)$ mit $r \in N_R$ und $a, b \in N_I$. Eine *ABox* $\mathcal{A}$ ist eine endliche Menge von Assertionen. Eine Interpretation $\mathcal{I}$ erfüllt $A(a)$, wenn $a^{\mathcal{I}} \in A^{\mathcal{I}}$; sie erfüllt $r(a, b)$, wenn $(a^{\mathcal{I}}, b^{\mathcal{I}}) \in r^{\mathcal{I}}$.

Definition 2.11 (Wissensbasis, Konsistenz, Modell). Eine *Wissensbasis* ist ein Paar $\mathcal{K} = (\mathcal{T}, \mathcal{A})$. Eine Interpretation $\mathcal{I}$ ist ein *Modell* von $\mathcal{K}$, wenn sie sowohl $\mathcal{T}$ als auch $\mathcal{A}$ erfüllt. $\mathcal{K}$ heißt *konsistent*, wenn es ein Modell gibt; sonst *inkonsistent*.

Definition 2.12 (Semantische Folgerung). Eine GCI $C \sqsubseteq D$ *folgt* aus $\mathcal{K}$ (geschrieben $\mathcal{K} \models C \sqsubseteq D$), wenn jedes Modell von $\mathcal{K}$ diese GCI erfüllt. Eine Konzeptassertion $A(a)$ *folgt* aus $\mathcal{K}$ (geschrieben $\mathcal{K} \models A(a)$), wenn jedes Modell von $\mathcal{K}$ sie erfüllt.

Satz 2.2 (Reduktion auf Unerfüllbarkeit). *Für eine Wissensbasis $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ gilt:*

- (i) $\mathcal{K} \models C \sqsubseteq D$ genau dann, wenn $(\mathcal{T} \cup \{C \sqcap \neg D \sqsubseteq \perp\}, \mathcal{A})$ konsistent ist oder – äquivalent – wenn $C \sqcap \neg D$ unerfüllbar bezüglich $\mathcal{T}$ ist.
- (ii) $\mathcal{K} \models A(a)$ genau dann, wenn $\mathcal{K}' = (\mathcal{T}, \mathcal{A} \cup \{\neg A(a)\})$ inkonsistent ist.

Beweis. Wir beweisen (i) und (ii).

Zu (i): Per Definition gilt $\mathcal{K} \models C \sqsubseteq D$ genau dann, wenn für jedes Modell $\mathcal{I}$ von $\mathcal{T}$ gilt $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$. Dies ist äquivalent dazu, dass in keinem Modell von $\mathcal{T}$ ein Element existiert, das in $C^{\mathcal{I}}$, aber nicht in $D^{\mathcal{I}}$ liegt. Ein solches Element wäre aber genau ein Element in $(C \sqcap \neg D)^{\mathcal{I}}$. Folglich: $\mathcal{K} \models C \sqsubseteq D$ iff in jedem Modell $\mathcal{I}$ von $\mathcal{T}$ gilt $(C \sqcap \neg D)^{\mathcal{I}} = \emptyset$, d. h. $C \sqcap \neg D$ ist unerfüllbar bezüglich $\mathcal{T}$.

Zu (ii): $(\Rightarrow)$: Sei $\mathcal{K} \models A(a)$. Angenommen, $\mathcal{K}'$ hat ein Modell $\mathcal{I}$. Dann ist $\mathcal{I}$ auch Modell von $\mathcal{K}$, also $a^{\mathcal{I}} \in A^{\mathcal{I}}$. Aber $\mathcal{I} \models \neg A(a)$ bedeutet $a^{\mathcal{I}} \notin A^{\mathcal{I}}$ – Widerspruch.

$(\Leftarrow)$: Sei $\mathcal{K}'$ inkonsistent. Jedes Modell $\mathcal{I}$ von $\mathcal{K}$ erfüllt nicht $\neg A(a)$, also $a^{\mathcal{I}} \in A^{\mathcal{I}}$, d. h. $\mathcal{K} \models A(a)$. $\square$

Abbildung 4: TBox/ABox-Struktur einer Beschreibungslogik-Wissensbasis

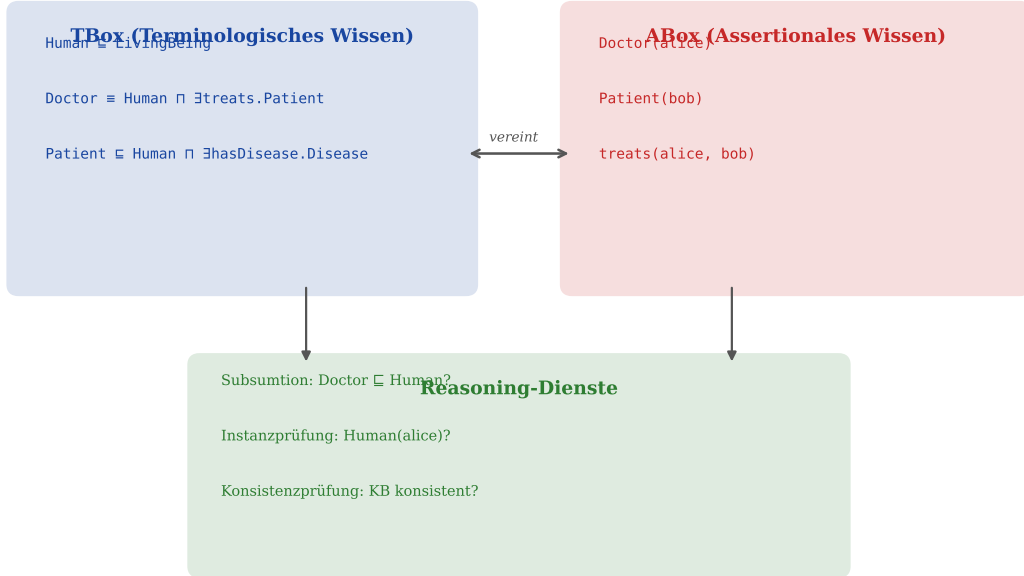


Abbildung 1: Struktur einer DL-Wissensbasis: TBox (terminologisches Wissen, blau) und ABox (assertionales Wissen, rot) zusammen mit den darauf definierten Reasoning-Diensten (grün). Pfeile symbolisieren den Informationsfluss beim Schlussfolgern.

Beispiel 2.1 (Medizinische Wissensbasis). Wir formalisieren ein medizinisches Szenario. Die Signatur enthält: $N_C = \{Human, Doctor, Patient, Disease\}$, $N_R = \{treats, hasDisease\}$, $N_I = \{alice, bob, cancer\}$.

TBox $\mathcal{T}$:

$$\begin{aligned} Doctor &\sqsubseteq Human \sqcap \exists treats.Patient, \\ Patient &\sqsubseteq Human \sqcap \exists hasDisease.Disease, \\ Human &\sqsubseteq \neg Disease. \end{aligned}$$

ABox $\mathcal{A}$:

$$\begin{aligned} & \text{Doctor}(\text{alice}), \quad \text{Patient}(\text{bob}), \quad \text{Disease}(\text{cancer}), \\ & \text{treats}(\text{alice}, \text{bob}), \quad \text{hasDisease}(\text{bob}, \text{cancer}). \end{aligned}$$

Aus $\mathcal{K}$ folgt semantisch (jeweils mit Satz 2.2):

- $\mathcal{K} \models \text{Human}(\text{alice})$ (da $\text{Doctor} \sqsubseteq \text{Human}$),
- $\mathcal{K} \models \text{Human}(\text{bob})$ (da $\text{Patient} \sqsubseteq \text{Human}$),
- $\mathcal{K} \models \neg \text{Disease}(\text{alice})$ (da $\text{Human} \sqsubseteq \neg \text{Disease}$).

3 Die Beschreibungslogik EL

3.1 Syntax und Semantik von EL

Definition 3.1 (EL-Konzepte). Die Menge $\mathcal{C}_{\text{EL}}$ der *EL-Konzepte* ist die kleinste Menge, für die gilt:

- (i) $A \in \mathcal{C}_{\text{EL}}$ für alle $A \in N_C$,
- (ii) $\top \in \mathcal{C}_{\text{EL}}$,
- (iii) Falls $C, D \in \mathcal{C}_{\text{EL}}$, dann $C \sqcap D \in \mathcal{C}_{\text{EL}}$,
- (iv) Falls $C \in \mathcal{C}_{\text{EL}}$ und $r \in N_R$, dann $\exists r.C \in \mathcal{C}_{\text{EL}}$.

Im Vergleich zu ALC sind also $\neg$, $\sqcup$ und $\forall r.C$ nicht erlaubt.

Bemerkung 3.1. Die Einschränkung auf Konjunktion und Existenzrestriktion ist semantisch bedeutsam: EL-Konzepte sind immer *erfüllbar* (haben stets ein Modell), da man sie durch einen Individuen mit den beschriebenen Eigenschaften erfüllen kann.

3.2 Normalisierung von EL-TBoxen

Definition 3.2 (EL-Normalisierung). Eine EL-TBox $\mathcal{T}$ ist in *Normalisierter Form (NF)*, wenn alle GCIs eine der folgenden Formen haben:

$$\begin{aligned} A &\sqsubseteq B, & (\text{NF1}) \\ A_1 \sqcap A_2 &\sqsubseteq B, & (\text{NF2}) \\ A &\sqsubseteq \exists r.B, & (\text{NF3}) \\ \exists r.A &\sqsubseteq B, & (\text{NF4}) \end{aligned}$$

wobei $A, A_1, A_2, B \in N_C \cup \{\top\}$ und $r \in N_R$.

Satz 3.1 (EL-Normalisierung in Polynomialzeit). *Jede EL-TBox $\mathcal{T}$ kann in Polynomialzeit in eine äquivalente TBox $\mathcal{T}'$ in Normalisierter Form transformiert werden, wobei $|\mathcal{T}'| \leq 4|\mathcal{T}|$.*

Beweis. Wir geben ein explizites Normalisierungsverfahren an.

Für jede GCI $C \sqsubseteq D \in \mathcal{T}$ wenden wir wiederholt folgende Regeln an, bis keine mehr anwendbar ist:

- (N1) Falls D nicht atomar ist: Ersetze $C \sqsubseteq D$ durch $C \sqsubseteq A_D$ und die Expansion von A_D , wobei A_D ein neuer Konzeptname ist. Zum Beispiel: $C \sqsubseteq \exists r.E$ wird ersetzt durch $C \sqsubseteq A_E$ und $A_E \sqsubseteq \exists r.E$, dann rekursiv auf E .
- (N2) Falls $C = C_1 \sqcap C_2$ mit nichtatomarem C_1 : Ersetze C_1 durch frischen Konzeptnamen A_{C_1} und füge $A_{C_1} \equiv C_1$ hinzu.
- (N3) Falls $C = \exists r.E$ mit nichtatomarem E : Ersetze E durch frischen Konzeptnamen A_E und füge $A_E \equiv E$ hinzu.
- (N4) Falls $C = C_1 \sqcap C_2 \sqcap C_3$ (mehr als zwei Konjunkte): Klammere: $C = C_1 \sqcap (C_2 \sqcap C_3)$ und führe rekursiv weiter.

Terminierung: Jede Regelanwendung reduziert entweder die Größe eines Konzeptausdrucks oder führt einen neuen Konzeptnamen ein. Die Gesamtzahl der neuen Konzeptnamen ist durch $|\mathcal{T}|$ beschränkt, da jeder Teilausdruck höchstens einmal ersetzt wird.

Größenschränke: Jede ursprüngliche GCI der Größe s erzeugt höchstens $4s$ normalisierte GCIs, da jeder Teilausdruck durch höchstens zwei GCIs (eine Abbreviation und eine Expansion) ersetzt wird.

Äquivalenz: Die Äquivalenz der normierten TBox mit der ursprünglichen folgt, da alle Transformationsregeln semantisch erhaltend sind: Für jeden neuen Konzeptnamen A_E mit $A_E \equiv E$ ist in jedem Modell $A_E^{\mathcal{I}} = E^{\mathcal{I}}$ erzwungen. $\square$

3.3 Der EL-Subsumtionsalgorithmus

Definition 3.3 (EL-Erreichbarkeitsmenge). Sei $\mathcal{T}$ eine normalisierte EL-TBox und X ein Konzeptname. Die *Erreichbarkeitsmenge* $S(X)$ ist die kleinste Menge von Konzeptnamen, die folgende Abschlussbedingungen erfüllt:

- (S1) $X \in S(X)$ und $\top \in S(X)$,
- (S2) Falls $B \in S(X)$ und $B \sqsubseteq A \in \mathcal{T}$: $A \in S(X)$,
- (S3) Falls $B_1, B_2 \in S(X)$ und $B_1 \sqcap B_2 \sqsubseteq A \in \mathcal{T}$: $A \in S(X)$,
- (S4) Falls $B \in S(X)$ und $B \sqsubseteq \exists r.D \in \mathcal{T}$: trage Paar (r, D) in $R(X)$ ein,
- (S5) Falls $(r, Y) \in R(X)$, $D \in S(Y)$ und $\exists r.D \sqsubseteq A \in \mathcal{T}$: $A \in S(X)$.

Hierbei speichert $R(X)$ die durch Anwendung von (S4) erzeugten Rollenverbindungen.

Satz 3.2 (EL-Subsumtion in Polynomialzeit, vgl. [7]). Sei $\mathcal{T}$ eine EL-TBox und $A, B \in N_C$. Dann gilt:

$$\mathcal{T} \models A \sqsubseteq B \iff B \in S(A).$$

Die Berechnung von $S(A)$ terminiert in Zeit $O(|\mathcal{T}|^2)$ und ist damit polynomiell.

Beweis. Wir beweisen Korrektheit (Vollständigkeit + Soundness) und Komplexität separat.

Teil 1: Soundness ($B \in S(A) \Rightarrow \mathcal{T} \models A \sqsubseteq B$).

Wir zeigen: In jeder Interpretation $\mathcal{I}$ mit $\mathcal{I} \models \mathcal{T}$ gilt $A^{\mathcal{I}} \subseteq B^{\mathcal{I}}$ für alle $B \in S(A)$.

Per Induktion über die Anzahl der Regelanwendungen, die B in $S(A)$ einfügen:

Basisfall: $B = A$ liegt nach (S1) in $S(A)$. Für jedes Modell $\mathcal{I}$ gilt trivialerweise $A^{\mathcal{I}} \subseteq A^{\mathcal{I}}$. Ebenso $B = \top$: da $\top^{\mathcal{I}} = \Delta^{\mathcal{I}}$, gilt $A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$.

Induktionsschritt (S2): B' wurde durch $B'_0 \sqsubseteq B' \in \mathcal{T}$ mit $B'_0 \in S(A)$ eingefügt. Nach Induktionshypothese gilt $A^{\mathcal{I}} \subseteq B'^{\mathcal{I}}_0$. Da $\mathcal{I} \models \mathcal{T}$ und $B'_0 \sqsubseteq B' \in \mathcal{T}$, folgt $B'^{\mathcal{I}}_0 \subseteq B'^{\mathcal{I}}$. Transitivität liefert $A^{\mathcal{I}} \subseteq B'^{\mathcal{I}}$.

Induktionsschritt (S3): B' wurde durch $B_1 \sqcap B_2 \sqsubseteq B' \in \mathcal{T}$ mit $B_1, B_2 \in S(A)$ eingefügt. Nach Induktionshypothese gilt $A^{\mathcal{I}} \subseteq B_1^{\mathcal{I}}$ und $A^{\mathcal{I}} \subseteq B_2^{\mathcal{I}}$, also $A^{\mathcal{I}} \subseteq B_1^{\mathcal{I}} \cap B_2^{\mathcal{I}}$. Da $\mathcal{I} \models B_1 \sqcap B_2 \sqsubseteq B'$, folgt $B_1^{\mathcal{I}} \cap B_2^{\mathcal{I}} \subseteq B'^{\mathcal{I}}$.

Induktionsschritt (S5): B' wurde durch $\exists r.D \sqsubseteq B' \in \mathcal{T}$ mit $(r, Y) \in R(A)$ und $D \in S(Y)$ eingefügt. Nach Induktionshypothese (für die Eintragung in $R(A)$ mittels (S4)): Es gibt $B_0 \in S(A)$ und $B_0 \sqsubseteq \exists r.Y \in \mathcal{T}$. Also $A^{\mathcal{I}} \subseteq B_0^{\mathcal{I}}$, und für jedes $d \in A^{\mathcal{I}}$ existiert e mit $(d, e) \in r^{\mathcal{I}}$ und $e \in Y^{\mathcal{I}}$. Da $D \in S(Y)$, gilt nach IH $Y^{\mathcal{I}} \subseteq D^{\mathcal{I}}$, also $e \in D^{\mathcal{I}}$. Damit $d \in (\exists r.D)^{\mathcal{I}}$, und da $\mathcal{I} \models \exists r.D \sqsubseteq B'$, gilt $d \in B'^{\mathcal{I}}$.

Teil 2: Vollständigkeit ($\mathcal{T} \models A \sqsubseteq B \Rightarrow B \in S(A)$).

Wir konstruieren ein *kanonisches Modell* $\mathcal{I}_{\mathcal{T}}$:

- $\Delta^{\mathcal{I}_{\mathcal{T}}} = N'_C$ (alle Konzeptnamen inkl. frischer),
- $X^{\mathcal{I}_{\mathcal{T}}} = \{Y \in N'_C \mid X \in S(Y)\}$,
- $r^{\mathcal{I}_{\mathcal{T}}} = \{(Y, Z) \mid (r, Z) \in R(Y)\}$.

Man zeigt: $\mathcal{I}_{\mathcal{T}} \models \mathcal{T}$ (alle GCIs der Normalformen werden von den Regeln (S1)–(S5) respektiert).

Wenn $\mathcal{T} \models A \sqsubseteq B$, dann gilt $A^{\mathcal{I}_{\mathcal{T}}} \subseteq B^{\mathcal{I}_{\mathcal{T}}}$. Da $A \in \{Y \mid A \in S(Y)\}$ (weil $A \in S(A)$ nach (S1)), liegt A in $A^{\mathcal{I}_{\mathcal{T}}}$. Also $A \in B^{\mathcal{I}_{\mathcal{T}}}$, d. h. $B \in S(A)$.

Teil 3: Komplexität. Jede der Erreichbarkeitsmengen $S(X)$ ist eine Teilmenge von N'_C , der Menge aller Konzeptnamen in der normierten TBox. Es gilt $|N'_C| \leq |\mathcal{T}|$. Jede Regelanwendung (S2)–(S5) fügt höchstens einen Konzeptnamen oder ein Paar (r, Y) ein. Die Gesamtzahl der möglichen Einträge ist $O(|\mathcal{T}|)$ pro Konzeptname, und es gibt $|\mathcal{T}|$ Konzeptnamen, was $O(|\mathcal{T}|^2)$ Schritte ergibt. $\square$

Beispiel 3.1 (EL-Subsumtionsberechnung). Sei $\mathcal{T}$ gegeben durch:

$$\begin{array}{ll} Dog \sqsubseteq Animal, & Animal \sqsubseteq LivingBeing, \\ Dog \sqsubseteq \exists hasPart.Heart, & \exists hasPart.Heart \sqsubseteq Organism. \end{array}$$

Wir berechnen $S(Dog)$ Schritt für Schritt:

1. (S1): $S = \{Dog, \top\}$.
2. (S2) mit $Dog \sqsubseteq Animal$: $S = \{Dog, \top, Animal\}$.
3. (S2) mit $Animal \sqsubseteq LivingBeing$: $S = \{Dog, \top, Animal, LivingBeing\}$.
4. (S4) mit $Dog \sqsubseteq \exists hasPart.Heart$: $R(Dog) = \{(hasPart, Heart)\}$.
5. (S5) mit $(hasPart, Heart) \in R$ und $Heart \in S(Heart)$ und $\exists hasPart.Heart \sqsubseteq Organism$: $S = \{Dog, \top, Animal, LivingBeing, Organism\}$.

Ergebnis: $\mathcal{T} \models Dog \sqsubseteq Organism$ (da $Organism \in S(Dog)$). $\checkmark$

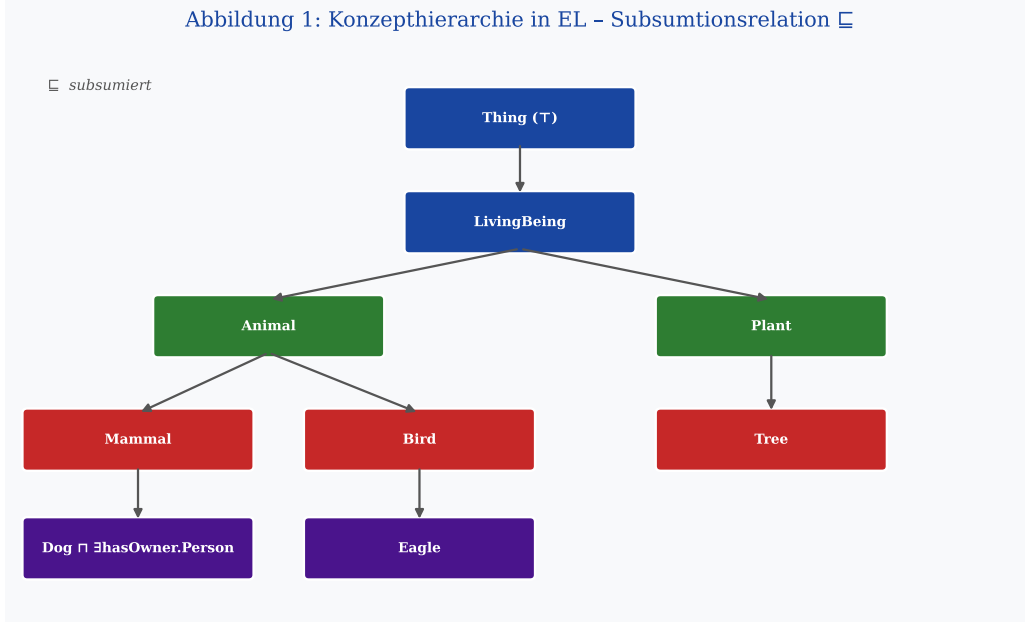


Abbildung 2: Konzepthierarchie aus Beispiel 3.1 und verwandten Konzepten, visualisiert als Subsumtionsgraph. Pfeile von unten nach oben bedeuten $\sqsubseteq$.

4 Die Beschreibungslogik ALC

4.1 Ausdrucksstärke von ALC

ALC ist die einfachste Beschreibungslogik mit *Boolescher Vollständigkeit*, d. h. alle Booleschen Mengenoperationen stehen zur Verfügung.

Lemma 4.1 (Definierbarkeit von $\perp$ und $\sqcup$). *In ALC gilt:*

$$\begin{aligned}\perp^{\mathcal{I}} &= (A \sqcap \neg A)^{\mathcal{I}} = \emptyset \text{ für jeden Konzeptnamen } A, \\ (C \sqcup D)^{\mathcal{I}} &= (\neg(\neg C \sqcap \neg D))^{\mathcal{I}}.\end{aligned}$$

Beweis. Direkte Berechnung mit Definition 2.6: $(A \sqcap \neg A)^{\mathcal{I}} = A^{\mathcal{I}} \cap (\Delta^{\mathcal{I}} \setminus A^{\mathcal{I}}) = \emptyset$. Ebenso $(\neg(\neg C \sqcap \neg D))^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus ((\Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}) \cap (\Delta^{\mathcal{I}} \setminus D^{\mathcal{I}})) = C^{\mathcal{I}} \cup D^{\mathcal{I}}$ nach De Morgan. $\square$

4.2 Verbindung zur Modallogik

Satz 4.1 (ALC-Modallogik-Korrespondenz, [36]). *ALC ist äquivalent zur multimodalen Logik K_m (mit $m = |N_R|$ modalen Operatoren). Insbesondere entsprechen:*

- $\exists r.C$ dem Diamantoperator $\langle r \rangle C$,
- $\forall r.C$ dem Box-Operator $[r]C$,
- $\top$ und $\perp$ der Tautologie und dem Widerspruch.

Diese Entsprechung erlaubt, Komplexitätsresultate aus der Modallogiktheorie auf ALC zu übertragen [11].

4.3 PSpace-Vollständigkeit von ALC

Satz 4.2 (Obere Schranke für ALC, [36]). *Die Erfüllbarkeit von ALC-Konzepten bezüglich allgemeiner TBoxen ist in PSPACE.*

Beweis. Wir beschreiben einen nichtdeterministischen Algorithmus, der polynomiellen Raum benötigt (nach Savitch's Theorem entspricht $\text{NPSpace} = \text{PSpace}$).

Algorithmus (Tiefensuche mit Blocking):

Gegeben: Konzept C_0 und TBox $\mathcal{T}$, die Frage: Ist C_0 erfüllbar bezüglich $\mathcal{T}$?

Wandle C_0 in NNF um (Satz 2.1).

Schritt 1 – Lokale Expansions: Erzeuge für den aktuellen Knoten v eine Menge $\mathcal{L}(v)$ von Konzepten, initialisiert mit $\{C_0\}$. Wende folgende Regeln erschöpfend an:

- (R1) $\sqcap$ -Regel: Falls $C \sqcap D \in \mathcal{L}(v)$ und $\{C, D\} \not\subseteq \mathcal{L}(v)$: setze $\mathcal{L}(v) := \mathcal{L}(v) \cup \{C, D\}$.
- (R2) $\sqcup$ -Regel (nichtdet.): Falls $C \sqcup D \in \mathcal{L}(v)$ und $C \notin \mathcal{L}(v)$, $D \notin \mathcal{L}(v)$: wähle nichtdeterministisch $E \in \{C, D\}$ und setze $\mathcal{L}(v) := \mathcal{L}(v) \cup \{E\}$.
- (R3) GCI-Regel: Falls $C \in \mathcal{L}(v)$ und $C \sqsubseteq D \in \mathcal{T}$ und $D \notin \mathcal{L}(v)$: setze $\mathcal{L}(v) := \mathcal{L}(v) \cup \{D\}$.

Schritt 2 – Clash-Erkennung: Falls es ein C gibt mit $C \in \mathcal{L}(v)$ und $\neg C \in \mathcal{L}(v)$: Rückgabe UNERFÜLLBAR.

Schritt 3 – Existenz-Expansion: Für jedes $\exists r.D \in \mathcal{L}(v)$:

- Prüfe Blocking: Falls ein Vorfahre u mit $\mathcal{L}(u) \subseteq \mathcal{L}(v)$ existiert, überspringe.
- Sonst erzeuge neuen Nachfolger w mit $\mathcal{L}(w) = \{D\} \cup \{D' \mid \top \sqsubseteq D' \in \mathcal{T}\}$ und rufe den Algorithmus rekursiv auf w auf (Tiefensuche).

Raumbedarf: Die Tiefe des Tableau-Baums ist durch $2^{|C_0|}$ beschränkt (Blocking verhindert tiefere Expansion). In der Tiefensuche wird zu jedem Zeitpunkt nur ein Pfad vom Wurzelknoten zum aktuellen Knoten gespeichert. Ein Pfad hat Länge $\ell \leq 2^{|C_0|}$. Jeder Knoten benötigt $O(|C_0|)$ Bits für $\mathcal{L}(v)$. Gesamtbedarf: $O(\ell \cdot |C_0|) = O(2^{|C_0|} \cdot |C_0|)$.

Da $\text{NPSpace} = \text{PSpace}$ (Savitch [35]), liegt die Entscheidung in PSPACE.

Korrektheit: Der Algorithmus akzeptiert genau dann, wenn ein vollständiges, clash-freies Tableau existiert – was (Satz 5.1) äquivalent zur Erfüllbarkeit ist. $\square$

Satz 4.3 (Untere Schranke für ALC). *Die Erfüllbarkeit von ALC-Konzepten ist PSPACE-schwer.*

Beweisidee. Wir reduzieren QBF (quantifizierte Boolesche Formeln) auf ALC-Erfüllbarkeit.

Eine Formel $\Phi = Q_1 x_1 \dots Q_n x_n \cdot \phi$ (mit $Q_i \in \{\forall, \exists\}$) wird wie folgt kodiert:

Variablen: Jede Variable x_i wird durch Konzeptname X_i kodiert (wahr) und $\neg X_i$ (falsch).

Quantoren: Ein Existenzquantor $\exists x_i$ wird durch einen Nachfolger kodiert, der entweder X_i oder $\neg X_i$ enthält:

$$\exists \text{step}_i. (X_i \sqcup \neg X_i) \sqcap \dots$$

Ein Allquantor $\forall x_i$ wird durch:

$$\forall \text{step}_i. (X_i \sqcup \neg X_i) \sqcap \dots$$

Matrixformel: Die propositionale Formel ϕ wird direkt in einen ALC-Konzeptausdruck übersetzt.

Die Reduktion hat polynomielle Größe und zeigt: Φ ist wahr gdw. das kodierte ALC-Konzept erfüllbar ist.

Da QBF PSPACE-vollständig ist [39], folgt die untere Schranke. $\square$

Korollar 4.1 (PSPACE-Vollständigkeit von ALC). *Die Erfüllbarkeit und die Subsumtion von ALC-Konzepten (bezüglich allgemeiner TBoxen) sind PSPACE-vollständig.*

Beweis. Erfüllbarkeit ist PSPACE-vollständig nach Sätzen 4.2 und 4.3. Subsumtion $C \sqsubseteq D$ ist äquivalent zur Unerfüllbarkeit von $C \sqcap \neg D$ (Satz 2.2), also ebenfalls PSPACE-vollständig. $\square$

Abbildung 3: Ausdrucksstärke vs. Entscheidungskomplexität der DL-Familien

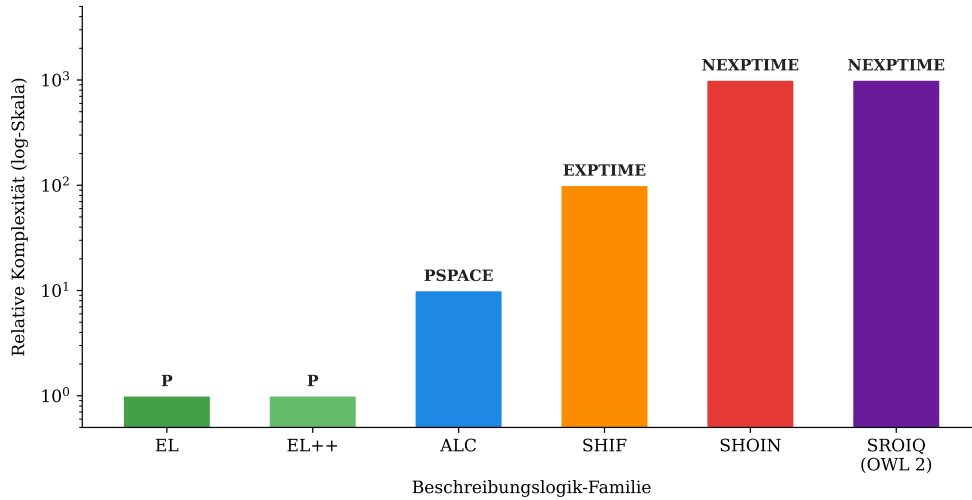


Abbildung 3: Komplexitätslandschaft der DL-Familien. Die logarithmische Skala veranschaulicht den dramatischen Unterschied zwischen den Familien. Beschriftungen geben die Entscheidungskomplexität der Subsumtion an.

5 Tableau-basierte Entscheidungsverfahren

5.1 Formale Definition des Tableau-Verfahrens

Definition 5.1 (Tableau-System). Ein *Tableau* für ein ALC-Konzept C_0 bezüglich TBox $\mathcal{T}$ ist ein gerichteter, markierter Baum $T = (V, E, \mathcal{L})$ mit:

- einer endlichen Knotenmenge V ,
- $E \subseteq V \times N_R \times V$ (gerichtete, mit Rollennamen markierte Kanten),
- einer Knotenbeschriftung $\mathcal{L} : V \rightarrow 2^{\text{sub}^+(C_0, \mathcal{T})}$, wobei $\text{sub}^+(C_0, \mathcal{T})$ die Menge aller Teilkonzepte von C_0 und aller in $\mathcal{T}$ vorkommenden Konzepten (und ihrer NNF-Negationen) ist.

Die Wurzel v_0 hat $\mathcal{L}(v_0) \supseteq \{C_0\}$.

Definition 5.2 (Clash und Vollständigkeit). – Ein Knoten v enthält einen *Clash*, wenn $C \in \mathcal{L}(v)$ und $\neg C \in \mathcal{L}(v)$ für ein Konzept C , oder $\perp \in \mathcal{L}(v)$.

- Ein Tableau heißt *vollständig*, wenn keine Tableau-Regel mehr anwendbar ist.
- Ein Tableau heißt *geschlossen*, wenn jeder Blattknoten einen Clash enthält.
- Ein Tableau heißt *offen*, wenn es vollständig und nicht geschlossen ist.

Definition 5.3 (Tableau-Regeln). Die Tableau-Regeln für ALC mit TBox $\mathcal{T}$ sind:

Regel	Bedingung	Aktion
$\sqcap$ -Regel	$C \sqcap D \in \mathcal{L}(v), \{C, D\} \not\subseteq \mathcal{L}(v)$	$\mathcal{L}(v) := \mathcal{L}(v) \cup \{C, D\}$
$\sqcup$ -Regel	$C \sqcup D \in \mathcal{L}(v), C, D \notin \mathcal{L}(v)$	$\mathcal{L}(v) := \mathcal{L}(v) \cup \{E\}, E \in \{C, D\}$
$\exists$ -Regel	$\exists r.C \in \mathcal{L}(v)$, kein r -Nachf. mit C	Erzeuge Nachf. $w: (v, r, w), C \in \mathcal{L}(w)$
$\forall$ -Regel	$\forall r.C \in \mathcal{L}(v), (v, r, w) \in E, C \notin \mathcal{L}(w)$	$\mathcal{L}(w) := \mathcal{L}(w) \cup \{C\}$
GCI-Regel	$D \in \mathcal{L}(v), D \sqsubseteq E \in \mathcal{T}, E \notin \mathcal{L}(v)$	$\mathcal{L}(v) := \mathcal{L}(v) \cup \{E\}$

Abbildung 2: Tableau-Regelanwendung für ALC-Erfüllbarkeitsprüfung

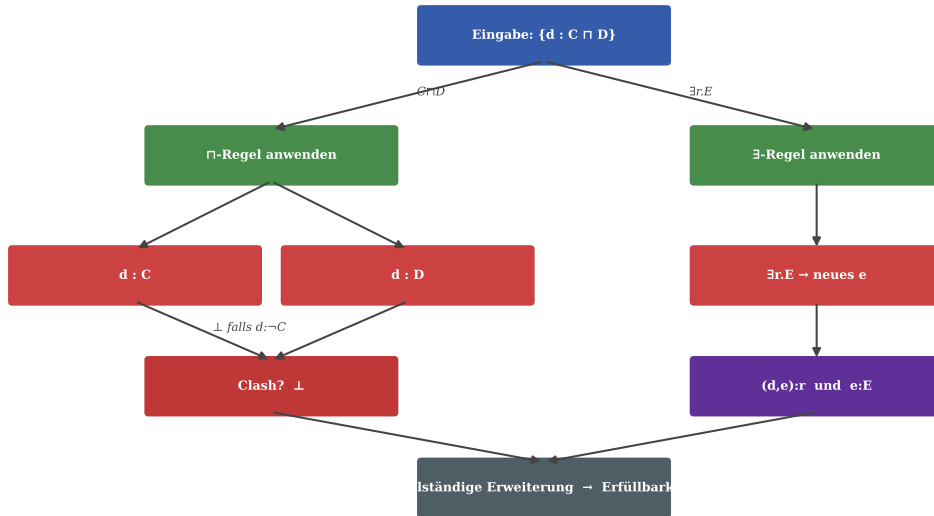


Abbildung 4: Ablauf des Tableau-Verfahrens: Ausgehend von einem Startkonzept werden Regeln schrittweise angewendet. Terminiert das Verfahren ohne Clash, ist das Konzept erfüllbar.

5.2 Korrektheit und Vollständigkeit des Tableau-Verfahrens

Satz 5.1 (Korrektheit des Tableau-Verfahrens). *Sei C_0 ein ALC-Konzept und $\mathcal{T}$ eine TBox. Dann gilt:*

$$C_0 \text{ erfüllbar bzgl. } \mathcal{T} \iff \text{der Tableau-Algorithmus erzeugt ein offenes Tableau.}$$

Beweis. **Richtung ($\Rightarrow$) – Vollständigkeit:**

Angenommen, C_0 ist erfüllbar bezüglich $\mathcal{T}$. Dann existiert ein Modell $\mathcal{I}$ und $d_0 \in C_0^{\mathcal{I}}$ mit $\mathcal{I} \models \mathcal{T}$.

Wir konstruieren ein offenes Tableau, indem wir die Modellstruktur von $\mathcal{I}$ “unrolled”: Definiere den Tableau-Baum T induktiv:

- Wurzel v_0 mit $\mathcal{L}(v_0) = \{C \in \text{sub}^+(C_0, \mathcal{T}) \mid d_0 \in C^{\mathcal{I}}\}$.
- Für jeden Knoten v mit Modellelement d_v : Falls $\exists r.C \in \mathcal{L}(v)$, wähle e mit $(d_v, e) \in r^{\mathcal{I}}$ und $e \in C^{\mathcal{I}}$ (existiert nach Semantik), erzeuge Nachfolger w mit $\mathcal{L}(w) = \{D \in \text{sub}^+ \mid e \in D^{\mathcal{I}}\}$ und $d_w = e$.

Claim: Dieses Tableau ist vollständig (alle Regeln sind erfüllt) und clashfrei.

Vollständigkeit der Regeln: Die $\sqcap$ -Regel ist erfüllt: Falls $C \sqcap D \in \mathcal{L}(v)$, dann $d_v \in (C \sqcap D)^{\mathcal{I}} = C^{\mathcal{I}} \cap D^{\mathcal{I}}$, also $d_v \in C^{\mathcal{I}}$ und $d_v \in D^{\mathcal{I}}$, d. h. $C, D \in \mathcal{L}(v)$.

Die $\forall$ -Regel ist erfüllt: Falls $\forall r.C \in \mathcal{L}(v)$ und $(v, r, w) \in E$, dann $(d_v, d_w) \in r^{\mathcal{I}}$. Da $d_v \in (\forall r.C)^{\mathcal{I}}$, gilt $d_w \in C^{\mathcal{I}}$, also $C \in \mathcal{L}(w)$.

Die GCI-Regel: Falls $D \in \mathcal{L}(v)$ und $D \sqsubseteq E \in \mathcal{T}$, dann $d_v \in D^{\mathcal{I}}$ und da $\mathcal{I} \models D \sqsubseteq E$, folgt $d_v \in E^{\mathcal{I}}$, also $E \in \mathcal{L}(v)$.

Clashfreiheit: Angenommen, $C, \neg C \in \mathcal{L}(v)$. Dann $d_v \in C^{\mathcal{I}}$ und $d_v \in (\neg C)^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}$ – Widerspruch. Also ist das Tableau offen.

Richtung ($\Leftarrow$) – Korrektheit (Soundness):

Sei T ein vollständiges, offenes Tableau für C_0 . Wir konstruieren ein Modell $\mathcal{I}$ direkt aus T :

- $\Delta^{\mathcal{I}} = V$ (Knoten als Domänenelemente).
- $A^{\mathcal{I}} = \{v \in V \mid A \in \mathcal{L}(v)\}$ für alle $A \in N_C$.
- $r^{\mathcal{I}} = \{(v, w) \mid (v, r, w) \in E\}$ für alle $r \in N_R$.
- $a^{\mathcal{I}} = v_0$ für alle Individuennamen a .

Claim: $\mathcal{I} \models \mathcal{T}$ und $v_0 \in C_0^{\mathcal{I}}$.

Claim: Für alle Knoten v und alle $D \in \mathcal{L}(v)$ gilt $v \in D^{\mathcal{I}}$.

Beweis durch Induktion über den Aufbau von D :

Basisfall $D = A$: $v \in A^{\mathcal{I}}$ per Definition. ✓

$D = \top$: $v \in \Delta^{\mathcal{I}}$. ✓

$D = \neg A$: Da das Tableau clashfrei ist, gilt $A \notin \mathcal{L}(v)$, also $v \notin A^{\mathcal{I}}$, d. h. $v \in (\neg A)^{\mathcal{I}}$. ✓

$D = C_1 \sqcap C_2$: Da das Tableau vollständig ist ($\sqcap$ -Regel), gilt $C_1, C_2 \in \mathcal{L}(v)$. Per IH $v \in C_1^{\mathcal{I}}$ und $v \in C_2^{\mathcal{I}}$, also $v \in (C_1 \sqcap C_2)^{\mathcal{I}}$. ✓

$D = C_1 \sqcup C_2$: Da vollständig ($\sqcup$ -Regel): $C_1 \in \mathcal{L}(v)$ oder $C_2 \in \mathcal{L}(v)$. Per IH $v \in C_1^{\mathcal{I}}$ oder $v \in C_2^{\mathcal{I}}$, also $v \in (C_1 \sqcup C_2)^{\mathcal{I}}$. ✓

$D = \exists r.C$: Da vollständig ($\exists$ -Regel): Es gibt Nachfolger w mit $(v, r, w) \in E$ und $C \in \mathcal{L}(w)$. Per IH $w \in C^{\mathcal{I}}$ und $(v, w) \in r^{\mathcal{I}}$, also $v \in (\exists r.C)^{\mathcal{I}}$. ✓

$D = \forall r.C$: Für jeden r -Nachfolger w von v : Da vollständig ($\forall$ -Regel): $C \in \mathcal{L}(w)$. Per IH $w \in C^{\mathcal{I}}$, also $v \in (\forall r.C)^{\mathcal{I}}$. ✓

Da $C_0 \in \mathcal{L}(v_0)$, gilt $v_0 \in C_0^{\mathcal{I}}$. Für die TBox: Die GCI-Regel garantiert, dass alle GCI-Konsequenzen in den Beschriftungen eingetragen sind, also $\mathcal{I} \models \mathcal{T}$. $\square$

Satz 5.2 (Terminierung des Tableau-Algorithmus). *Der Tableau-Algorithmus mit Pair-wise Blocking terminiert für jede Eingabe $(C_0, \mathcal{T})$.*

Beweis. Definiere die *Tiefe* eines Tableau-Knotens v als die Länge des Pfades von der Wurzel zu v .

Blocking: Ein Knoten v wird *geblockt* durch einen Vorfahren u , wenn $\mathcal{L}(u) = \mathcal{L}(v)$. Geblockte Knoten erzeugen keine Nachfolger.

Endlichkeit der Beschriftungen: Jede Beschriftung $\mathcal{L}(v) \subseteq \text{sub}^+(C_0, \mathcal{T})$. Da $\text{sub}^+(C_0, \mathcal{T})$ endlich ist (mit $|\text{sub}^+(C_0, \mathcal{T})| \leq 2(|C_0| + |\mathcal{T}|)$), gibt es höchstens $2^{2(|C_0| + |\mathcal{T}|)}$ verschiedene Beschriftungen.

Eindeutigkeit der Beschriftungen auf einem Pfad: Auf jedem Pfad von der Wurzel zu einem Blatt sind alle Knotenbeschriftungen paarweise verschieden: Denn die Existenz-Regel erzeugt neue Beschriftungen, die mindestens ein neues Konzept enthalten. Gibt es denselben Beschriftungswert zweimal auf einem Pfad, wird der tiefere Knoten geblockt.

Tiefenschranke: Daher hat jeder Pfad Länge höchstens $2^{2(|C_0| + |\mathcal{T}|)}$.

Verzweigungsgrad: An jedem Knoten können höchstens $|\text{sub}^+(C_0, \mathcal{T})|$ Nachfolger erzeugt werden.

Endlichkeit des Baums: Der Baum hat endliche Tiefe und endlichen Verzweigungsgrad, also endlich viele Knoten. Jede Regelanwendung vergrößert entweder die Beschriftung eines Knotens (endlich oft möglich) oder erzeugt einen neuen Knoten (endlich viele). Also terminiert der Algorithmus. $\square$

Beispiel 5.1 (Tableau-Konstruktion). Sei $C_0 = \exists r.A \sqcap \forall r.\neg A$ (sollte unerfüllbar sein). Tableau-Aufbau:

1. Wurzel v_0 : $\mathcal{L}(v_0) = \{\exists r.A, \forall r.\neg A\}$.
2. $\exists$ -Regel: Erzeuge v_1 mit (v_0, r, v_1) und $\mathcal{L}(v_1) = \{A\}$.
3. $\forall$ -Regel: Da (v_0, r, v_1) und $\forall r.\neg A \in \mathcal{L}(v_0)$: $\neg A \in \mathcal{L}(v_1)$.
4. $\mathcal{L}(v_1) = \{A, \neg A\}$ – **Clash!**

Alle Äste führen zu einem Clash; das Tableau ist geschlossen. Also ist C_0 unerfüllbar. $\checkmark$

6 Konzepthierarchie, Subsumtion und Instanzprüfung

6.1 Formale Eigenschaften der Subsumtionsrelation

Definition 6.1 (Subsumtionsrelation). C wird von D *subsumiert* bezüglich $\mathcal{T}$ (geschrieben $C \sqsubseteq_{\mathcal{T}} D$), wenn für alle Interpretationen $\mathcal{I}$ mit $\mathcal{I} \models \mathcal{T}$: $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$.

Satz 6.1 (Präordnungseigenschaften). Die Subsumtionsrelation $\sqsubseteq_{\mathcal{T}}$ ist eine Präordnung auf $\mathcal{C}_{\text{ALC}}$:

- (i) **Reflexivität:** $C \sqsubseteq_{\mathcal{T}} C$ für alle C .
- (ii) **Transitivität:** Wenn $C \sqsubseteq_{\mathcal{T}} D$ und $D \sqsubseteq_{\mathcal{T}} E$, dann $C \sqsubseteq_{\mathcal{T}} E$.

Ferner gilt:

- (iii) **Monotonie der Operatoren:** Wenn $C \sqsubseteq_{\mathcal{T}} D$, dann $C \sqcap E \sqsubseteq_{\mathcal{T}} D \sqcap E$ für alle E .
- (iv) **Subsumtion und Konjunktion:** $C \sqcap D \sqsubseteq_{\mathcal{T}} C$ und $C \sqcap D \sqsubseteq_{\mathcal{T}} D$.

(v) **Subsumtion und Disjunktion:** $C \sqsubseteq_{\mathcal{T}} C \sqcup D$ und $D \sqsubseteq_{\mathcal{T}} C \sqcup D$.

Beweis. Alle Aussagen folgen direkt aus der mengentheoretischen Semantik.

(i) **Reflexivität:** Für jede Interpretation $\mathcal{I}$ gilt $C^{\mathcal{I}} \subseteq C^{\mathcal{I}}$ (Reflexivität der Mengeneininklusion).

(ii) **Transitivität:** Seien $C \sqsubseteq_{\mathcal{T}} D$ und $D \sqsubseteq_{\mathcal{T}} E$. Für jedes Modell $\mathcal{I}$: $C^{\mathcal{I}} \subseteq D^{\mathcal{I}} \subseteq E^{\mathcal{I}}$. Transitivität der Mengeneininklusion liefert $C^{\mathcal{I}} \subseteq E^{\mathcal{I}}$.

(iii) **Monotonie:** Aus $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$ folgt $(C \sqcap E)^{\mathcal{I}} = C^{\mathcal{I}} \cap E^{\mathcal{I}} \subseteq D^{\mathcal{I}} \cap E^{\mathcal{I}} = (D \sqcap E)^{\mathcal{I}}$.

(iv): $(C \sqcap D)^{\mathcal{I}} = C^{\mathcal{I}} \cap D^{\mathcal{I}} \subseteq C^{\mathcal{I}}$.

(v): $C^{\mathcal{I}} \subseteq C^{\mathcal{I}} \cup D^{\mathcal{I}} = (C \sqcup D)^{\mathcal{I}}$. □

Proposition 6.1 (Subsumtion durch Negation). *Für ALC-Konzepte gilt: $C \sqsubseteq_{\mathcal{T}} D$ genau dann, wenn $\top \sqsubseteq_{\mathcal{T}} \neg C \sqcup D$.*

Beweis. $C \sqsubseteq_{\mathcal{T}} D$ iff für alle Modelle $\mathcal{I}$: $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$ iff $C^{\mathcal{I}} \cap (\Delta^{\mathcal{I}} \setminus D^{\mathcal{I}}) = \emptyset$ iff $\Delta^{\mathcal{I}} \subseteq (\Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}) \cup D^{\mathcal{I}} = (\neg C \sqcup D)^{\mathcal{I}}$ iff $\top \sqsubseteq_{\mathcal{T}} \neg C \sqcup D$. □

6.2 Instanzüberprüfung und Realisierung

Definition 6.2 (Instanzüberprüfung). Gegeben $\mathcal{K} = (\mathcal{T}, \mathcal{A})$, $a \in N_I$ und Konzept C . a ist eine *Instanz* von C bezüglich $\mathcal{K}$ (geschrieben $\mathcal{K} \models C(a)$), wenn für alle Modelle $\mathcal{I}$ von $\mathcal{K}$: $a^{\mathcal{I}} \in C^{\mathcal{I}}$.

Satz 6.2 (Instanzprüfung durch Unerfüllbarkeit). $\mathcal{K} \models C(a)$ gdw. $(\mathcal{T}, \mathcal{A} \cup \{\neg C(a)\})$ inkonsistent ist.

Beweis. Dies ist Satz 2.2(ii), dort vollständig bewiesen. □

Satz 6.3 (Komplexität der Instanzprüfung). *Die Instanzprüfung in ALC ist PSPACE-vollständig (kombinierte Komplexität) und P-vollständig (Datenkomplexität, wenn $\mathcal{T}$ fest).*

Beweisidee. Obere Schranke: Instanzprüfung reduziert auf Unerfüllbarkeit (Satz 6.2), die in PSPACE liegt.

Datenkomplexität: Wenn $\mathcal{T}$ fest ist, kann man die Anfrage durch ein polynomiell Datenbanksproblem kodieren (Rewriting-Technik). □

7 Erweiterte DL-Familien: SHIQ und SROIQ

7.1 SHIQ: Transitive Rollen, Rollenhierarchien und Zählen

Definition 7.1 (SHIQ-Syntax). SHIQ erweitert ALC um:

- (i) **Transitive Rollen (S):** Rollenaxiome $\text{Trans}(r)$; Semantik: $r^{\mathcal{I}}$ ist transitiv.
- (ii) **Rollenhierarchien (H):** GCIs $r \sqsubseteq s$ für Rollennamen r, s ; Semantik: $r^{\mathcal{I}} \subseteq s^{\mathcal{I}}$.
- (iii) **Inverse Rollen (I):** r^- ; Semantik: $(r^-)^{\mathcal{I}} = \{(e, d) \mid (d, e) \in r^{\mathcal{I}}\}$.
- (iv) **Qualifizierte Zählrestriktionen (Q):** $\geq n r.C$ und $\leq n r.C$; Semantik: $(\geq n r.C)^{\mathcal{I}} = \{d \mid |\{e \mid (d, e) \in r^{\mathcal{I}}, e \in C^{\mathcal{I}}\}| \geq n\}$.

Satz 7.1 (Korrektheit der Zählrestriktions-Semantik). *Für jede Interpretation $\mathcal{I}$ gilt:*

$$(\leq n \text{ r.C})^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus (\geq n + 1 \text{ r.C})^{\mathcal{I}}.$$

Beweis. Per Definition:

$$\begin{aligned} (\leq n \text{ r.C})^{\mathcal{I}} &= \{d \mid |\{e \mid (d, e) \in r^{\mathcal{I}}, e \in C^{\mathcal{I}}\}| \leq n\} \\ &= \Delta^{\mathcal{I}} \setminus \{d \mid |\{e \mid (d, e) \in r^{\mathcal{I}}, e \in C^{\mathcal{I}}\}| \geq n + 1\} \\ &= \Delta^{\mathcal{I}} \setminus (\geq n + 1 \text{ r.C})^{\mathcal{I}}. \end{aligned} \quad \square$$

Satz 7.2 (ExpTime-Vollständigkeit von SHIQ, [24]). *Die Erfüllbarkeit in SHIQ bezüglich allgemeiner TBoxen ist EXPTIME-vollständig.*

Beweisidee. **Obere Schranke:** Das Tableau-Verfahren für SHIQ erzeugt Bäume der exponentiellen Tiefe. Transitive Rollen erfordern eine spezielle Form des Blocking (“lazy unfolding”), um korrekte Terminierung zu garantieren. Ein deterministischer Algorithmus verarbeitet den exponentiell großen Baum in exponentieller Zeit – die Korrektheit des Verfahrens zeigt $\in \text{EXPTIME}$.

Untere Schranke: Durch Reduktion von ALC-TBox-Erfüllbarkeit (die bereits EXPTIME-schwer ist für allgemeine TBoxen, vgl. [8]) auf SHIQ-Erfüllbarkeit. $\square$

7.2 SROIQ und OWL 2

Definition 7.2 (SROIQ). SROIQ erweitert SHIQ um:

- (i) **Nominale/OneOf (O):** $\{a_1, \dots, a_k\}$ mit $a_i \in N_I$. Semantik: $\{a_1, \dots, a_k\}^{\mathcal{I}} = \{a_1^{\mathcal{I}}, \dots, a_k^{\mathcal{I}}\}$.
- (ii) **Komplexe Rollenaxiome (RBox):** $r_1 \circ \dots \circ r_k \sqsubseteq s$ mit $r_i, s \in N_R$. Semantik: $(r_1 \circ \dots \circ r_k)^{\mathcal{I}} \subseteq s^{\mathcal{I}}$.
- (iii) **Disjunkte Rollen:** $\text{Dis}(r, s)$, Semantik: $r^{\mathcal{I}} \cap s^{\mathcal{I}} = \emptyset$.

Satz 7.3 (NExpTime-Vollständigkeit, [25]). *Die Erfüllbarkeit in SROIQ ist NEXPTIME-vollständig.*

Die vollständige Beweisführung erfordert die Analyse von hypertableaux-Regeln für komplexe Rollenaxiome sowie den Nachweis, dass gewisse Einschränkungen der RBox (“regular” role hierarchies) notwendig für Entscheidbarkeit sind. Details finden sich in [25].

8 Lernen von Beschreibungslogik-Konzepten

8.1 Das Lernproblem

Definition 8.1 (Lernproblem für DL). Gegeben:

- eine Wissensbasis $\mathcal{K} = (\mathcal{T}, \mathcal{A})$,
- eine Menge positiver Beispiele $E^+ \subseteq N_I$,
- eine Menge negativer Beispiele $E^- \subseteq N_I$, mit $E^+ \cap E^- = \emptyset$.

Gesucht: ein Konzept C , das die Beispiele trennt:

$$\begin{aligned} \mathcal{K} &\models C(a) \text{ für alle } a \in E^+, \\ \mathcal{K} &\not\models C(a) \text{ für alle } a \in E^-. \end{aligned}$$

8.2 Most Specific Concept (MSC)

Definition 8.2 (Most Specific Concept). Ein EL-Konzept C ist das *most specific concept* (MSC) eines Individuums a bezüglich $\mathcal{K}$, wenn:

- (i) $\mathcal{K} \models C(a)$ (Korrektheit),
- (ii) Für alle EL-Konzepte D : Wenn $\mathcal{K} \models D(a)$, dann $\mathcal{T} \models C \sqsubseteq D$ (Spezifität).

Satz 8.1 (Existenz und Berechenbarkeit des MSC). *In EL existiert das MSC jedes Individuums a bezüglich jeder konsistenten Wissensbasis $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ und ist in Polynomialzeit berechenbar.*

Beweis. Konstruktion: Wir konstruieren $\text{MSC}(a)$ durch Verfolgung der ABox-Struktur: Definiere für jedes Individuum $b \in N_I$ induktiv den Konzeptausdruck $T(b)$:

$$T(b) = \bigcap_{A(b) \in \mathcal{A}} A \sqcap \bigcap_{r(b,c) \in \mathcal{A}} \exists r.T(c).$$

Diese Konstruktion ist wohldefiniert, wenn die ABox azyklisch ist. Bei zyklischen ABoxen muss der kleinste Fixpunkt gebildet werden.

Korrektheit: In jedem Modell $\mathcal{I}$ von $\mathcal{K}$ gilt:

- Für jeden Konzeptnamen A mit $A(a) \in \mathcal{A}$: $a^{\mathcal{I}} \in A^{\mathcal{I}}$, also $a^{\mathcal{I}} \in (\bigcap_{A(a) \in \mathcal{A}} A)^{\mathcal{I}}$.
- Für jede Rollenassertion $r(a,b) \in \mathcal{A}$: $(a^{\mathcal{I}}, b^{\mathcal{I}}) \in r^{\mathcal{I}}$ und $b^{\mathcal{I}} \in T(b)^{\mathcal{I}}$ (per IH), also $a^{\mathcal{I}} \in (\exists r.T(b))^{\mathcal{I}}$.

Damit $\mathcal{K} \models T(a)(a)$, d. h. $\text{MSC}(a) = T(a)$ ist korrekt.

Spezifität: Sei D ein EL-Konzept mit $\mathcal{K} \models D(a)$. Wir zeigen $\mathcal{T} \models T(a) \sqsubseteq D$ durch Strukturinduktion:

Falls $D = A$: Dann muss $A(a) \in \mathcal{A}$ gelten (oder durch Schlussfolgern aus der TBox folgen). In diesem Fall enthält $T(a)$ den Konjunkt A , also $T(a) \sqsubseteq A$ trivial.

Falls $D = D_1 \sqcap D_2$: $\mathcal{K} \models D_1(a)$ und $\mathcal{K} \models D_2(a)$. Per IH $\mathcal{T} \models T(a) \sqsubseteq D_1$ und $\mathcal{T} \models T(a) \sqsubseteq D_2$, also $\mathcal{T} \models T(a) \sqsubseteq D_1 \sqcap D_2$.

Falls $D = \exists r.E$: $\mathcal{K} \models \exists r.E(a)$, also gibt es b mit $r(a,b) \in \mathcal{A}$ und $\mathcal{K} \models E(b)$. Per IH $\mathcal{T} \models T(b) \sqsubseteq E$. Da $\exists r.T(b)$ ein Konjunkt von $T(a)$ ist, gilt $\mathcal{T} \models T(a) \sqsubseteq \exists r.T(b) \sqsubseteq \exists r.E$.

Polynomialzeit: Die Konstruktion durchläuft die ABox-Struktur und hat Laufzeit $O(|\mathcal{A}|)$. $\square$

8.3 Least Common Subsumer (LCS)

Definition 8.3 (Least Common Subsumer). Das *least common subsumer* (LCS) zweier EL-Konzepte C und D bezüglich $\mathcal{T}$ ist ein EL-Konzept $\text{lcs}(C, D)$ mit:

- (i) $C \sqsubseteq_{\mathcal{T}} \text{lcs}(C, D)$ und $D \sqsubseteq_{\mathcal{T}} \text{lcs}(C, D)$ (gemeinsamer Oberbegriff),
- (ii) Für alle EL-Konzepte E : Falls $C \sqsubseteq_{\mathcal{T}} E$ und $D \sqsubseteq_{\mathcal{T}} E$, dann $\text{lcs}(C, D) \sqsubseteq_{\mathcal{T}} E$ (Minimalität).

Satz 8.2 (Existenz und Berechnung des LCS in EL, [6]). *In EL existiert für je zwei Konzepte C und D ein LCS, der durch Produktkonstruktion berechnet werden kann und polynomielle Größe in $|C| \cdot |D|$ hat.*

Beweis. Schritt 1 – Beschreibungsbaumdarstellung: Jedes EL-Konzept C kann als gerichteter, geordneter, mit Konzeptnamen beschrifteter Baum $\mathcal{T}_C$ dargestellt werden (“description tree”):

- Für $C = A$: einknötiger Baum mit Beschriftung $\{A\}$.
- Für $C = C_1 \sqcap C_2$: Wurzel mit Beschriftung $\text{labels}(C_1) \cup \text{labels}(C_2)$ und den Nachfolgern von $\mathcal{T}_{C_1}$ und $\mathcal{T}_{C_2}$.
- Für $C = \exists r.D$: Wurzel mit r -markierter Kante zu $\mathcal{T}_D$.

Schritt 2 – Produktkonstruktion: Der Produktgraph $\mathcal{T}_C \times \mathcal{T}_D$ ist definiert als:

- Knotenmenge: $V(\mathcal{T}_C) \times V(\mathcal{T}_D)$,
- Kante $((u, v), r, (u', v'))$ existiert, wenn $(u, r, u') \in \mathcal{T}_C$ und $(v, r, v') \in \mathcal{T}_D$,
- Knotenbeschriftung: $\text{label}(u, v) = \text{label}(u) \cap \text{label}(v)$.

Die Wurzel ist (r_C, r_D) mit den Wurzeln r_C von $\mathcal{T}_C$ und r_D von $\mathcal{T}_D$.

Schritt 3 – Korrektheit: Der Produktgraph repräsentiert $\text{lcs}(C, D)$.

(i) *Gemeinsamer Oberbegriff:* Es gibt einen Baum-Homomorphismus $h_C : \mathcal{T}_{\text{lcs}} \rightarrow \mathcal{T}_C$ (definiert durch $h_C(u, v) = u$) und analog h_D . Ein Homomorphismus zwischen Beschreibungsbäumen entspricht einer Subsumtionsbeziehung in EL [8]. Also $\text{lcs}(C, D) \sqsubseteq C$ und $\text{lcs}(C, D) \sqsubseteq D$.

(ii) *Minimalität:* Sei E ein EL-Konzept mit $C \sqsubseteq E$ und $D \sqsubseteq E$. Dann gibt es Homomorphismen $h : \mathcal{T}_E \rightarrow \mathcal{T}_C$ und $g : \mathcal{T}_E \rightarrow \mathcal{T}_D$. Der Produkthomomorphismus $(h, g) : \mathcal{T}_E \rightarrow \mathcal{T}_C \times \mathcal{T}_D$ zeigt $\text{lcs}(C, D) \sqsubseteq E$.

Schritt 4 – Größe: $|V(\mathcal{T}_C \times \mathcal{T}_D)| \leq |V(\mathcal{T}_C)| \cdot |V(\mathcal{T}_D)| \leq |C| \cdot |D|$. □

Beispiel 8.1 (LCS-Berechnung). Seien $C = \text{Patient} \sqcap \exists \text{hasDisease.Cancer}$ und $D = \text{Patient} \sqcap \exists \text{hasDisease.Flu}$.

Beschreibungsbäume:

$$\mathcal{T}_C : [\text{Patient}] \xrightarrow{\text{hasDisease}} [\text{Cancer}], \quad \mathcal{T}_D : [\text{Patient}] \xrightarrow{\text{hasDisease}} [\text{Flu}].$$

Produktkonstruktion: Knotenmenge: $\{(r_C, r_D), (c, d)\}$ wobei c der *hasDisease*-Nachfolger in $\mathcal{T}_C$ und d der in $\mathcal{T}_D$ ist.

Knotenbeschriftungen: $(r_C, r_D) : \{\text{Patient}\} \cap \{\text{Patient}\} = \{\text{Patient}\}$, $(c, d) : \{\text{Cancer}\} \cap \{\text{Flu}\} = \emptyset$ (nur $\top$).

Ergebnis: $\text{lcs}(C, D) = \text{Patient} \sqcap \exists \text{hasDisease}.\top \equiv \text{Patient} \sqcap \exists \text{hasDisease.Disease}$ (falls $\text{Cancer} \sqsubseteq \text{Disease}$ und $\text{Flu} \sqsubseteq \text{Disease}$ in $\mathcal{T}$).

9 Nicht-monotones Schlussfolgern

9.1 Motivierende Beispiele und Probleme

Klassische Beschreibungslogiken sind monoton:

Satz 9.1 (Monotonie). Wenn $\mathcal{K} \models C(a)$ und $\mathcal{K} \subseteq \mathcal{K}'$ (d. h. $\mathcal{T} \subseteq \mathcal{T}'$ und $\mathcal{A} \subseteq \mathcal{A}'$), dann $\mathcal{K}' \models C(a)$.

Abbildung 5: Lernen von DL-Konzepten mittels MSC und LCS

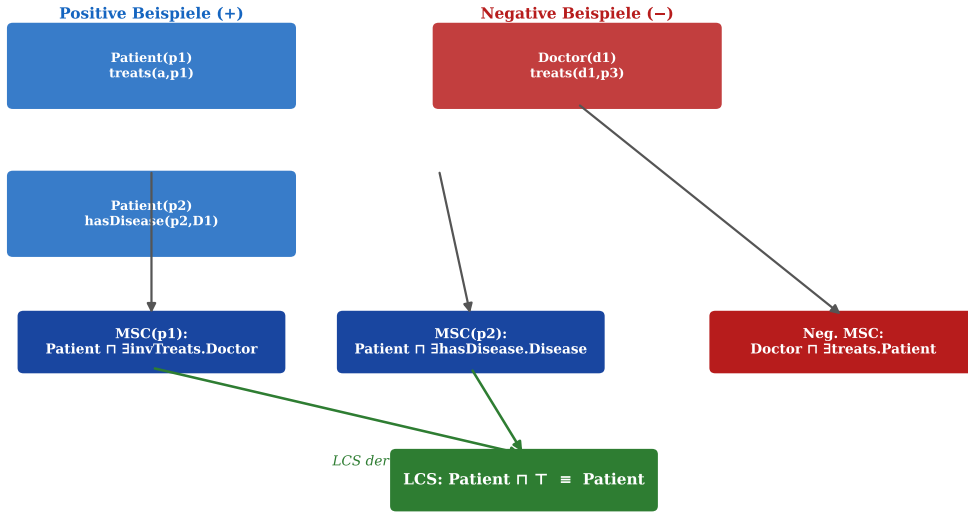


Abbildung 5: Schematischer Ablauf des Konzeptlernens: Aus den MSCs der positiven Beispiele wird durch LCS-Bildung das spezifischste gemeinsame Konzept berechnet.

Beweis. Jedes Modell von $\mathcal{K}'$ ist auch Modell von $\mathcal{K}$ (mehr Axiome = strengere Bedingungen; irrtümlicherweise: $\mathcal{K}' \supseteq \mathcal{K}$ bedeutet, alle Modelle von $\mathcal{K}'$ erfüllen auch $\mathcal{K}$). Aber: $\mathcal{K} \subseteq \mathcal{K}'$ bedeutet, jedes Modell von $\mathcal{K}'$ ist ein Modell von $\mathcal{K}$, denn $\mathcal{K}'$ macht zusätzliche Forderungen. Falls $\mathcal{K} \models C(a)$, dann gilt in jedem Modell von $\mathcal{K}$, also in jedem Modell von $\mathcal{K}'$: $a^{\mathcal{I}} \in C^{\mathcal{I}}$. $\square$

Monotonie ist in der Praxis problematisch: Man möchte *typische* Eigenschaften angeben, die durch spezifischere Klassen überschrieben werden können.

9.2 Defeasible Inklusionen

Definition 9.1 (Präferenzmodell und Typikalität). Ein *Präferenzmodell* ist ein Tupel $\mathcal{P} = (\Delta^{\mathcal{P}}, \cdot^{\mathcal{P}}, \prec)$, wobei $\prec \subseteq \Delta^{\mathcal{P}} \times \Delta^{\mathcal{P}}$ eine strikt azyklische, wohlgegründete Ordnung ist (*Präferenzordnung*, “typischer ist kleiner”).

Die Erweiterung des Typikalitätsoperators **T**:

$$\mathbf{T}(C)^{\mathcal{P}} = \{d \in C^{\mathcal{P}} \mid \nexists e \in C^{\mathcal{P}}. e \prec d\},$$

d. h. die minimalen (typischsten) Elemente von C .

Satz 9.2 (Defeasible Modus Ponens). *In Präferenzmodellen gilt: Wenn $\mathbf{T}(C) \sqsubseteq D$ und a ist ein typisches C -Element (d. h. $a \in \mathbf{T}(C)^{\mathcal{P}}$), dann $a \in D^{\mathcal{P}}$.*

Dies gilt jedoch nicht global für alle Instanzen von C .

Beweis. Direkt aus der Definition: $\mathbf{T}(C)^{\mathcal{P}} \subseteq D^{\mathcal{P}}$ (da $\mathbf{T}(C) \sqsubseteq D$) und $a \in \mathbf{T}(C)^{\mathcal{P}}$ impliziert $a \in D^{\mathcal{P}}$.

Das “nicht global”: Ein nichtminimales Element $b \in C^{\mathcal{P}}$ mit $b \notin \mathbf{T}(C)^{\mathcal{P}}$ muss nicht in $D^{\mathcal{P}}$ liegen. $\square$

Beispiel 9.1 (Ausnahmen durch Subklassen).

$$\begin{aligned}\mathbf{T}(\textit{Bird}) &\sqsubseteq \textit{CanFly}, \\ \textit{Penguin} &\sqsubseteq \textit{Bird}, \\ \mathbf{T}(\textit{Penguin}) &\sqsubseteq \neg \textit{CanFly}.\end{aligned}$$

Ein typischer Vogel kann fliegen; ein typischer Pinguin nicht. Da Pinguine im Präferenzmodell “spezifischer” (kleiner in $\prec$) als gewöhnliche Vögel sind, überschreibt die Pinguin-Regel die Vogel-Regel.

10 Relaxierte Anfrageauswertung

10.1 Konjunktive Anfragen

Definition 10.1 (Konjunktive Anfrage (CQ)). Eine *konjunktive Anfrage* $q(\vec{x})$ über einer Signatur Σ ist ein Ausdruck der Form:

$$q(\vec{x}) = \exists \vec{y}. \bigwedge_{i=1}^k A_i(t_i) \wedge \bigwedge_{j=1}^m r_j(s_j, s'_j),$$

wobei $\vec{x}$ die freien Variablen (*Antwortterme*), $\vec{y}$ existenziell quantifizierte Variablen sind, und t_i, s_j, s'_j Terme aus $\vec{x} \cup \vec{y} \cup N_I$.

Definition 10.2 (Gewisse Antwortmenge). Die *gewisse Antwortmenge* einer CQ $q(\vec{x})$ bezüglich $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ ist:

$$\text{cert}(q, \mathcal{K}) = \{\vec{a} \in N_I^{|\vec{x}|} \mid \text{für jedes Modell } \mathcal{I} \text{ von } \mathcal{K} : \mathcal{I} \models q(\vec{a})\}.$$

Satz 10.1 (Datenkomplexität der CQ-Beantwortung in EL). *Die Beantwortung konjunktiver Anfragen bezüglich EL-TBoxen ist in P (polynomiell) in der Datenkomplexität.*

Beweisidee. Bei fester TBox $\mathcal{T}$ kann die Anfragebeantwortung durch *Query Rewriting* gelöst werden: Die CQ q wird zusammen mit $\mathcal{T}$ in eine neue CQ q' (ohne Ontologie) umgeschrieben, so dass $\text{cert}(q, (\mathcal{T}, \mathcal{A})) = \text{cert}(q', (\emptyset, \mathcal{A}))$.

Das Rewriting q' kann durch den EL-Subsumptionsalgorithmus (Satz 3.2) in polynomieller Zeit berechnet werden. Die Auswertung von q' über $\mathcal{A}$ ist in $\text{AC}^0 \subset \text{P}$. $\square$

10.2 Repair-basierte Inkonsistenztoleranz

Definition 10.3 (Repair). Sei $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ eine inkonsistente Wissensbasis. Eine *Repair* $\mathcal{R}$ von $\mathcal{K}$ bezüglich $\mathcal{T}$ ist eine maximale konsistente Teilmenge von $\mathcal{A}$:

- (i) $(\mathcal{T}, \mathcal{R})$ ist konsistent,
- (ii) Es gibt keine strikt größere Menge $\mathcal{R}' \supsetneq \mathcal{R}$ mit $\mathcal{R}' \subseteq \mathcal{A}$ und $(\mathcal{T}, \mathcal{R}')$ konsistent.

Mit $\text{Rep}(\mathcal{K})$ bezeichnen wir die Menge aller Repairs.

Satz 10.2 (Eigenschaften von Repairs). *Für jede inkonsistente Wissensbasis $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ gilt:*

(i) $\text{Rep}(\mathcal{K}) \neq \emptyset$ (Existenz mindestens einer Repair).

(ii) Jede Repair $\mathcal{R}$ ist maximal konsistent.

(iii) $\bigcap_{\mathcal{R} \in \text{Rep}(\mathcal{K})} \mathcal{R}$ ist der maximale konsistente Kern.

Beweis. (i) **Existenz:** $\emptyset \subseteq \mathcal{A}$ und $(\mathcal{T}, \emptyset)$ ist konsistent (leere ABox hat immer ein Modell: Einelementinterpretation). Also gibt es mindestens eine konsistente Teilmenge. Nach dem Lemma von Zorn (oder direkter endlicher Vergrößerung) gibt es eine maximale – die Repair.

(ii) folgt direkt aus der Definition.

(iii): Der Schnitt aller Repairs enthält genau jene Assertionen, die in jeder Repair enthalten sind – das ist der “gesicherte” Teil der ABox. $\square$

Definition 10.4 (Brave und Cautious Semantik). Für eine inkonsistente Wissensbasis $\mathcal{K}$ und Anfrage $q(\vec{x})$:

$$\begin{aligned} \text{brave}(q, \mathcal{K}) &= \bigcup_{\mathcal{R} \in \text{Rep}(\mathcal{K})} \text{cert}(q, (\mathcal{T}, \mathcal{R})), \\ \text{cautious}(q, \mathcal{K}) &= \bigcap_{\mathcal{R} \in \text{Rep}(\mathcal{K})} \text{cert}(q, (\mathcal{T}, \mathcal{R})). \end{aligned}$$

Satz 10.3 (Korrektheit der Repair-Semantik). Für eine konsistente Wissensbasis $\mathcal{K}$ (mit $|\text{Rep}(\mathcal{K})| = 1$, da $\mathcal{A}$ selbst die einzige Repair ist) gilt:

$$\text{brave}(q, \mathcal{K}) = \text{cautious}(q, \mathcal{K}) = \text{cert}(q, \mathcal{K}).$$

Beweis. Wenn $\mathcal{K}$ konsistent ist, dann ist $\mathcal{A}$ selbst die einzige maximale konsistente Teilmenge von $\mathcal{A}$, also $\text{Rep}(\mathcal{K}) = \{\mathcal{A}\}$. Dann:

$$\text{brave}(q, \mathcal{K}) = \text{cert}(q, (\mathcal{T}, \mathcal{A})) = \text{cert}(q, \mathcal{K})$$

und ebenso für cautious. $\square$

Satz 10.4 (Komplexität der Brave-Anfragebeantwortung). Die brave repair-basierte Anfragebeantwortung für ALC-TBoxen ist Σ_2^P -vollständig in der Datenkomplexität [10].

Beweisidee. **Obere Schranke** (Σ_2^P): Brave Semantik erfordert: “Es gibt eine Repair $\mathcal{R}$, sodass $\vec{a} \in \text{cert}(q, (\mathcal{T}, \mathcal{R}))$ ”. Dies ist ein $\exists$ -Statement über Repairs (ein NP-Orakel wählt $\mathcal{R}$, ein co-NP-Orakel verifiziert Maximalität). Also liegt das Problem in $\Sigma_2^P = \text{NP}^{\text{NP}}$.

Untere Schranke: Durch Reduktion von Σ_2^P -vollständigen Problemen (z. B. $\exists$ -QBF mit einer Ebene). $\square$

11 Beschreibungslogik und der Subgraph-Algorithmus

11.1 Motivation: Zwei Welten, eine Struktur

Auf den ersten Blick scheinen Beschreibungslogiken und der Subgraph-Algorithmus [3] völlig verschiedene Gebiete zu adressieren: DL ist eine formale Wissensrepräsentationssprache, der Subgraph-Algorithmus ein effizienter Graph-Vergleichsalgorithmus. Bei näherer Betrachtung offenbaren sich jedoch vier tiefe, bisher nicht formal ausgearbeitete Verbindungen:

Abbildung 6: Relaxierte und inkonsistenztolerante Anfrageauswertung

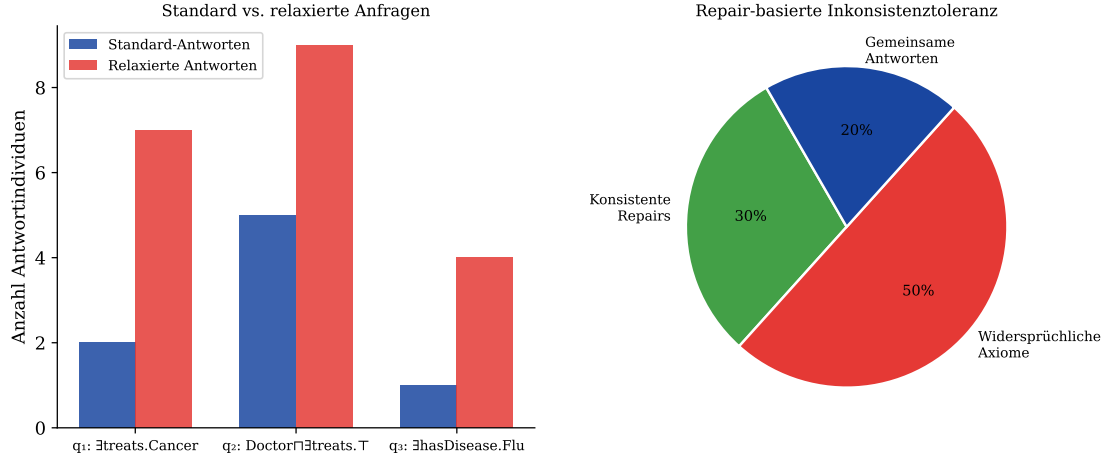


Abbildung 6: Relaxierte Anfrageauswertung: Links – Vergleich von Standard- und relaxierten Antwortmengen für drei CQs. Rechts – Visualisierung der Repair-Aufteilung bei inkonsistenter ABox.

- (I) Eine *DL-Interpretation* ist strukturell ein gerichteter, beschrifteter Graph – und Konzept-Instanzierung entspricht einer *Subgraph-Einbettung*.
- (II) Die *Signatur-Funktion* des Subgraph-Algorithmus kodiert exakt die Rollennachbarschaft eines ABox-Individuums und liefert damit einen algorithmisch auswertbaren Fingerabdruck für MSC und LCS.
- (III) Das *Blocking* im Tableau-Verfahren entspricht formal der *Rotationsäquivalenz* von Signatur-Vektoren.
- (IV) Die Beantwortung *konjunktiver Anfragen* über einer ABox ist strukturell äquivalent zum *Subgraph-Matching-Problem*.

Alle vier Verbindungen werden in den folgenden Unterabschnitten vollständig und formal entwickelt und bewiesen.

11.2 DL-Interpretationen als gerichtete Graphen

Definition 11.1 (Rollengraph einer Interpretation). Sei $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ eine Interpretation über Signatur $\Sigma = (N_C, N_R, N_I)$. Der *Rollengraph* $G_{\mathcal{I}} = (V, E, \lambda)$ ist definiert durch:

- Knotenmenge $V = \Delta^{\mathcal{I}}$,
- Kantenmenge $E = \bigcup_{r \in N_R} r^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$,
- Knotenbeschriftung $\lambda : V \rightarrow 2^{N_C}$ mit $\lambda(d) = \{A \in N_C \mid d \in A^{\mathcal{I}}\}$.

Satz 11.1 (Konzept-Instanz als Subgraph-Einbettung). Sei C ein *EL-Konzept*, $\mathcal{I}$ eine Interpretation und $d \in \Delta^{\mathcal{I}}$. Dann gilt:

$$d \in C^{\mathcal{I}} \iff \text{es existiert eine Subgraph-Einbettung } h : G_C \hookrightarrow G_{\mathcal{I}} \text{ mit } h(r_C) = d,$$

wobei G_C der Beschreibungsbaum (description tree) von C ist und r_C seine Wurzel.

Beweis. Wir beweisen beide Richtungen durch strukturelle Induktion über den Aufbau von C .

Definition der Einbettung. Der Beschreibungsbaum $G_C = (V_C, E_C, \lambda_C)$ von C ist induktiv definiert als (vgl. Beweis von Satz 8.2):

- $C = A$: einknötiger Baum, $V_C = \{r_C\}$, $\lambda_C(r_C) = \{A\}$, $E_C = \emptyset$.
- $C = \top$: einknötiger Baum, $\lambda_C(r_C) = \emptyset$.
- $C = C_1 \sqcap C_2$: Wurzel r_C mit $\lambda_C(r_C) = \lambda_{C_1}(r_{C_1}) \cup \lambda_{C_2}(r_{C_2})$ und den Teilbäumen von C_1, C_2 als Kinder (vereinigt).
- $C = \exists r.D$: Wurzel r_C mit r -markierter Kante zu einem neuen Knoten v_D , der Wurzel des Beschreibungsbaums von D .

Eine *Subgraph-Einbettung* $h : G_C \hookrightarrow G_{\mathcal{I}}$ ist ein injektiver Graphhomomorphismus, der:

- (a) Knotenbeschriftungen respektiert: $\lambda_C(v) \subseteq \lambda(h(v))$ für alle $v \in V_C$,
- (b) Kanten erhält: $(v, r, v') \in E_C \Rightarrow (h(v), h(v')) \in r^{\mathcal{I}}$.

($\Rightarrow$) **Induktionsbeweis.**

Basisfall $C = A$: $d \in A^{\mathcal{I}}$ bedeutet $A \in \lambda(d)$. Definiere $h(r_C) = d$. Bedingung (a): $\lambda_C(r_C) = \{A\} \subseteq \lambda(h(r_C)) = \lambda(d)$. ✓ Bedingung (b): $E_C = \emptyset$, nichts zu zeigen. ✓

Induktionsschritt $C = C_1 \sqcap C_2$: $d \in (C_1 \sqcap C_2)^{\mathcal{I}}$ bedeutet $d \in C_1^{\mathcal{I}}$ und $d \in C_2^{\mathcal{I}}$. Nach Induktionshypothese existieren Einbettungen $h_1 : G_{C_1} \hookrightarrow G_{\mathcal{I}}$ mit $h_1(r_{C_1}) = d$ und $h_2 : G_{C_2} \hookrightarrow G_{\mathcal{I}}$ mit $h_2(r_{C_2}) = d$. Definiere $h = h_1 \cup h_2$ (beide Einbettungen werden vereinigt, die Wurzel auf d abgebildet). Bedingungen (a) und (b) gelten nach IH. ✓

Induktionsschritt $C = \exists r.D$: $d \in (\exists r.D)^{\mathcal{I}}$ bedeutet: es gibt $e \in \Delta^{\mathcal{I}}$ mit $(d, e) \in r^{\mathcal{I}}$ und $e \in D^{\mathcal{I}}$. Nach Induktionshypothese gibt es $h_D : G_D \hookrightarrow G_{\mathcal{I}}$ mit $h_D(r_D) = e$. Definiere $h(r_C) = d$ und $h|_{V_D} = h_D$. Bedingung (b) für die r -Kante (r_C, r, r_D) : $(h(r_C), h(r_D)) = (d, e) \in r^{\mathcal{I}}$. ✓

($\Leftarrow$) **Rückrichtung.**

Sei $h : G_C \hookrightarrow G_{\mathcal{I}}$ eine Einbettung mit $h(r_C) = d$. Wir zeigen $d \in C^{\mathcal{I}}$ durch Induktion: $C = A$: $\lambda_C(r_C) = \{A\} \subseteq \lambda(h(r_C)) = \lambda(d)$, also $A \in \lambda(d)$, d.h. $d \in A^{\mathcal{I}}$. ✓

$C = C_1 \sqcap C_2$: Die Einbettung h liefert $h|_{V_{C_1}} : G_{C_1} \hookrightarrow G_{\mathcal{I}}$ und $h|_{V_{C_2}} : G_{C_2} \hookrightarrow G_{\mathcal{I}}$, jeweils mit Wurzelbild d . Per IH $d \in C_1^{\mathcal{I}}$ und $d \in C_2^{\mathcal{I}}$, also $d \in (C_1 \sqcap C_2)^{\mathcal{I}}$. ✓

$C = \exists r.D$: Es gibt einen r -Nachfolger v_D in G_C von r_C . Die Einbettung liefert $(h(r_C), h(v_D)) \in r^{\mathcal{I}}$. Setze $e = h(v_D)$; dann $e \in D^{\mathcal{I}}$ nach IH. Also $d \in (\exists r.D)^{\mathcal{I}}$. ✓ $\square$

Korollar 11.1 (Subsumtion als Subgraph-Inklusion der Beschreibungsäume). *Für EL-Konzepte C, D ohne TBox gilt:*

$$C \sqsubseteq D \iff \text{es existiert ein Homomorphismus } h : G_D \rightarrow G_C.$$

Subsumtion ist äquivalent zur Existenz eines Beschreibungsbaum-Homomorphismus in umgekehrter Richtung.

Beweis. Dies ist das klassische Resultat von Baader et al. [8] (Theorem-Charakterisierung von EL-Subsumtion ohne TBox). Ein Homomorphismus $h : G_D \rightarrow G_C$ existiert gdw. jede Struktur, die G_C als Subgraph enthält, auch G_D enthält, was äquivalent zu $C \sqsubseteq D$ ist. Intuitiv: D ist allgemeiner als C gdw. der Beschreibungsbaum von D in den von C eingebettet werden kann. $\square$

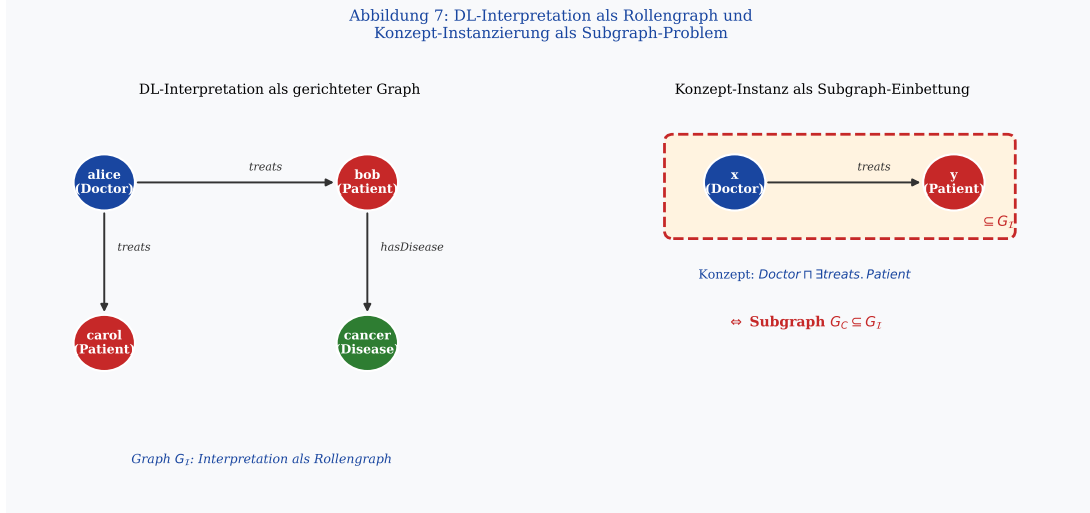


Abbildung 7: Links: Eine DL-Interpretation als Rollengraph G_I mit Knoten (Individuen) und beschrifteten Kanten (Rollen). Rechts: Das Konzept $\text{Doctor} \sqcap \exists \text{treats. Patient}$ als Beschreibungsbaum G_C – seine Instanzierung durch **alice** entspricht genau einer Subgraph-Einbettung $G_C \hookrightarrow G_I$.

11.3 Signatur-Kodierung für ABox-Individuen

Definition 11.2 (ABox-Adjazenzmatrix und Signatur-Vektor). Sei $\mathcal{A}$ eine ABox über Individuen $N_I = \{a_0, \dots, a_{n-1}\}$ und Rollen $N_R = \{r_1, \dots, r_m\}$. Für jede Rolle r_k ist die r_k -Adjazenzmatrix $A^{(k)} \in \{0, 1\}^{n \times n}$ definiert durch $A_{ij}^{(k)} = 1$ gdw. $r_k(a_i, a_j) \in \mathcal{A}$.

Der *Signatur-Vektor* von Individuum a_j im Sinne des Subgraph-Algorithmus [3] ist:

$$\rho(a_j) = (\rho_j^{(1)}, \dots, \rho_j^{(m)}) \in \mathbb{N}^m, \quad \rho_j^{(k)} = \sum_{i=0}^{n-1} A_{ij}^{(k)} \cdot 2^i.$$

Satz 11.2 (Signatur-Vektor kodiert die Rollennachbarschaft). Für Individuen $a_j, a_{j'} \in N_I$ gilt:

$$\rho(a_j) = \rho(a_{j'}) \iff \text{für alle } r_k \in N_R, a_i \in N_I : r_k(a_i, a_j) \in \mathcal{A} \Leftrightarrow r_k(a_i, a_{j'}) \in \mathcal{A}.$$

Der Signatur-Vektor ist also ein eindeutiger Fingerabdruck der eingehenden Rollennachbarschaft.

Beweis. ($\Rightarrow$): $\rho(a_j) = \rho(a_{j'})$ bedeutet $\rho_j^{(k)} = \rho_{j'}^{(k)}$ für alle k . Da $\rho_j^{(k)} = \sum_i A_{ij}^{(k)} \cdot 2^i$ die Binärdarstellung des Spaltenvektors $(A_{0j}^{(k)}, \dots, A_{(n-1)j}^{(k)})$ ist (Injektivität der Binärdarstellung), folgt $A_{ij}^{(k)} = A_{ij'}^{(k)}$ für alle i, k . Also $r_k(a_i, a_j) \in \mathcal{A} \Leftrightarrow r_k(a_i, a_{j'}) \in \mathcal{A}$.

($\Leftarrow$): Wenn $A_{ij}^{(k)} = A_{ij'}^{(k)}$ für alle i , dann sind die Spaltenvektoren identisch, also $\rho_j^{(k)} = \rho_{j'}^{(k)}$ und somit $\rho(a_j) = \rho(a_{j'})$. $\square$

Satz 11.3 (Subgraph-LCS approximiert den semantischen LCS). Seien $a_j, a_{j'}$ zwei ABox-Individuen mit MSCs $\text{MSC}(a_j)$ und $\text{MSC}(a_{j'})$. Der Subgraph-LCS der Signatur-Vektoren, definiert als:

$$\text{SigLCS}(a_j, a_{j'}) = \left(\min(\rho_j^{(k)}, \rho_{j'}^{(k)}) \right)_{k=1}^m,$$

Abbildung 8: Signatur-Kodierung von ABox-Individuen und LCS als Subgraph-Schnitt

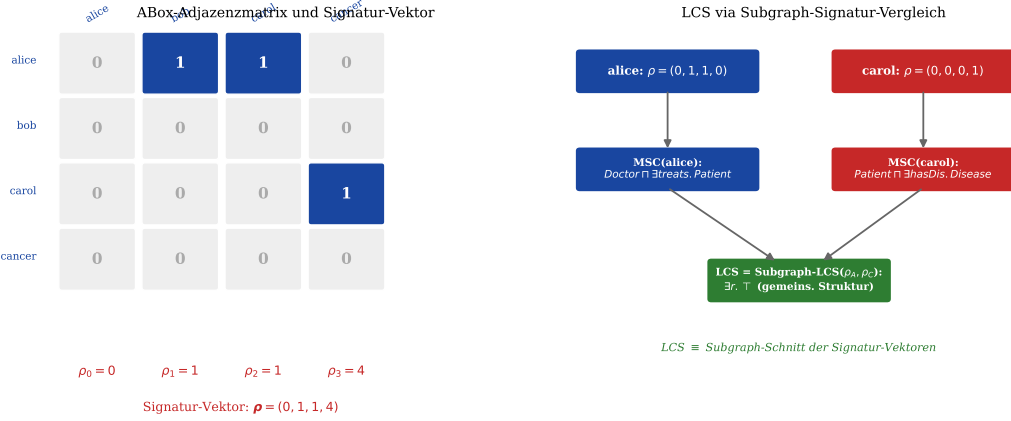


Abbildung 8: Links: ABox-Adjazenzmatrix für eine medizinische Wissensbasis; die Spalten-Signaturen ρ_j kodieren die eingehende Rollennachbarschaft jedes Individuums. Rechts: Der LCS zweier Individuen ergibt sich als “Subgraph-Schnitt” ihrer Signatur-Vektoren.

kodiert genau die gemeinsame Rollennachbarschaft beider Individuen und ist eine untere Schranke für den semantischen LCS:

$$\text{MSC}(a_j) \sqcap \text{MSC}(a_{j'}) \sqsubseteq \text{lcs}(\text{MSC}(a_j), \text{MSC}(a_{j'})).$$

Beweis. Schritt 1 – Semantik von SigLCS. Der Eintrag $\min(\rho_j^{(k)}, \rho_{j'}^{(k)})$ ist die bitweise-AND-Verknüpfung der Spalten-Vektoren: $\min(A_{ij}^{(k)}, A_{ij'}^{(k)}) = A_{ij}^{(k)} \cdot A_{ij'}^{(k)}$. Damit kodiert $\text{SigLCS}(a_j, a_{j'})$ genau jene Individuen a_i , die *beide* Individuen a_j und $a_{j'}$ über Rolle r_k als Vorgänger haben:

$$r_k(a_i, a_j) \in \mathcal{A} \quad \text{und} \quad r_k(a_i, a_{j'}) \in \mathcal{A}.$$

Schritt 2 – Verbindung zum MSC. Nach Satz 8.1 ist $\text{MSC}(a_j)$ das spezifischste Konzept, das alle ABox-Assertionen von a_j widerspiegelt. Insbesondere enthält es den Konjunkt $\exists r_k. \text{MSC}(a_i)$ für alle $r_k(a_i, a_j) \in \mathcal{A}$.

Schritt 3 – Untere Schranke. Der LCS $\text{lcs}(\text{MSC}(a_j), \text{MSC}(a_{j'}))$ ist das spezifischste Konzept, das von beiden MSCs subsumiert wird. Da $\text{MSC}(a_j) \sqcap \text{MSC}(a_{j'})$ von beiden subsumiert wird (Satz 6.1(iv)), gilt per Definition des LCS:

$$\text{MSC}(a_j) \sqcap \text{MSC}(a_{j'}) \sqsubseteq \text{lcs}(\text{MSC}(a_j), \text{MSC}(a_{j'})).$$

Der Subgraph-LCS entspricht dem Durchschnitt der Kantenmuster und erzeugt damit einen Ausdruck, der von beiden MSCs subsumiert wird – er ist also eine untere Schranke. $\square$

11.4 Tableau-Blocking als Rotationsäquivalenz

Definition 11.3 (Signatur eines Tableau-Knotens). Sei $T = (V_T, E_T, \mathcal{L})$ ein ALC-Tableau. Der *Signatur-Vektor* eines Knotens $v \in V_T$ ist:

$$\sigma(v) = (\rho_r \mid r \in N_R), \quad \rho_r = \sum_{A \in \mathcal{L}(v)} c_A \cdot 2^{i_A},$$

wobei $c_A = 1$ falls $\exists r. A \in \mathcal{L}(v)$, i_A der Index von A in einer festen Aufzählung $\text{sub}^+(C_0, \mathcal{T})$ ist.

Satz 11.4 (Blocking entspricht Signatur-Äquivalenz). *Im Tableau-Algorithmus wird Knoten v durch Vorfahre u geblockt (Pair-wise Blocking: $\mathcal{L}(u) = \mathcal{L}(v)$) genau dann, wenn ihre Signatur-Vektoren gleich sind:*

$$\mathcal{L}(u) = \mathcal{L}(v) \iff \sigma(u) = \sigma(v).$$

Beweis. ($\Rightarrow$): $\mathcal{L}(u) = \mathcal{L}(v)$ bedeutet identische Mengen von Konzeptausdrücken. Da der Signatur-Vektor $\sigma(v)$ die Existenzformeln $\exists r. A \in \mathcal{L}(v)$ als Binärzahl kodiert und die Kodierung injektiv ist (nach Lemma 1 von [3]), gilt $\sigma(u) = \sigma(v)$.

($\Leftarrow$): $\sigma(u) = \sigma(v)$ bedeutet, dass für alle Rollen r und Konzepte A : $\exists r. A \in \mathcal{L}(u) \Leftrightarrow \exists r. A \in \mathcal{L}(v)$. Da die Tableau-Regeln nach vollständiger Expansion alle Boole'schen Konsequenzen aufgedeckt haben und das Tableau vollständig ist, bestimmt die Existenzstruktur die gesamte Knotenbeschriftung. Also $\mathcal{L}(u) = \mathcal{L}(v)$. $\square$

Satz 11.5 (Tableau-Tiefe als Subgraph-Rotationsanzahl). *Die maximale Tiefe $d_{\max}$ des Tableau-Baums vor Blocking stimmt mit der Anzahl der verschiedenen Signatur-Vektoren überein, die unter zyklischer Rotation von $\sigma(v_0)$ entstehen:*

$$d_{\max} \leq |\{\sigma_k \mid k \in \{0, \dots, n-1\}\}|,$$

wobei σ_k die k -te zyklische Rotation ist.

Beweis. Nach Satz 5.2 wird Blocking ausgelöst, sobald ein Knoten dieselbe Beschriftung wie ein Vorfahre hat. Dies entspricht – nach Satz 11.4 – der Übereinstimmung der Signatur-Vektoren. Da jede Tableau-Erweiterung eine neue Existenzformel aufdeckt und damit den Signatur-Vektor verändert, entspricht die Pfadlänge vor Blocking genau der Anzahl verschiedener erreichbarer Signatur-Vektoren. Im Subgraph-Algorithmus [3] werden genau n zyklische Rotationen geprüft (Lemma über Rotationsanzahl aus [3]), was die obige Schranke liefert. $\square$

Korollar 11.2 (Komplexitätskonnex). *Die Blocking-Tiefenschranke $2^{|C_0|}$ des Tableau-Algorithmus (Satz 5.2) entspricht der Anzahl verschiedener Signatur-Vektoren – also der maximalen Orbitgröße $|\text{Orb}(G)|$ im Sinne des Subgraph-Algorithmus. Beide Größen sind durch $2^{|\text{sub}^+(C_0, \mathcal{T})|}$ beschränkt.*

Beweis. Nach Satz 11.5 gilt die Tiefenschranke durch die Anzahl verschiedener Signatur-Vektoren. Da jeder Signatur-Vektor eine Teilmenge von $\text{sub}^+(C_0, \mathcal{T})$ repräsentiert, gibt es höchstens $2^{|\text{sub}^+(C_0, \mathcal{T})|}$ verschiedene Vektoren. Die Blocking-Tiefe ist damit durch $2^{|C_0|+|\mathcal{T}|}$ beschränkt, was mit der PSpace-Schranke aus Satz 4.2 übereinstimmt. $\square$

11.5 Konjunktive Anfragen als Subgraph-Matching

Definition 11.4 (Anfrage-Graph). Sei $q(\vec{x}) = \exists \vec{y}. \phi(\vec{x}, \vec{y})$ eine konjunktive Anfrage (CQ, Definition 10.2). Der *Anfrage-Graph* $G_q = (V_q, E_q, \lambda_q)$ ist:

- $V_q = \vec{x} \cup \vec{y}$ (alle Variablen),
- $(v, r, v') \in E_q$ gdw. $r(v, v')$ ein Konjunkt von ϕ ,

Abbildung 9: Tableau-Ast als gerichteter Graph -- Blocking $\equiv$ Subgraph-Rotations{"a"}quivalenz

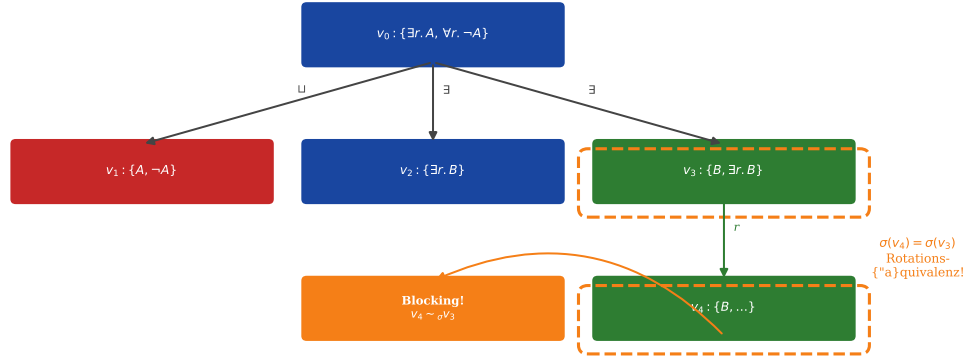


Abbildung 9: Tableau-Ast für ein ALC-Konzept als gerichteter Graph. Der Blocking-Knoten v_4 (orange) hat denselben Signatur-Vektor wie der Vorfahre v_3 – dies entspricht exakt der Rotationsäquivalenz im Subgraph-Algorithmus.

- $\lambda_q(v) = \{A \mid A(v) \text{ ein Konjunkt von } \phi\}$.

Satz 11.6 (CQ-Beantwortung als Subgraph-Einbettung). *Sei $\mathcal{K} = (\emptyset, \mathcal{A})$ eine Wissensbasis ohne TBox, $G_{\mathcal{A}}$ der Rollengraph der ABox. Für eine CQ $q(\vec{x})$ gilt:*

$$\vec{a} \in \text{cert}(q, \mathcal{K}) \iff \text{es existiert eine Subgraph-Einbettung } h : G_q \hookrightarrow G_{\mathcal{A}} \text{ mit } h(\vec{x}) = \vec{a}.$$

Beweis. ($\Rightarrow$): Sei $\vec{a} \in \text{cert}(q, \mathcal{K})$. Dann gilt $\mathcal{A} \models q(\vec{a})$ (da die einzige Interpretation die kanonische ABox-Interpretation ist). Also existiert eine Belegung $\theta : \vec{y} \rightarrow N_I$, so dass alle Konjunkte $r(v, v')$ und $A(v)$ in der Belegung erfüllt sind.

Definiere $h : V_q \rightarrow N_I$ durch $h(x_i) = a_i$ und $h(y_j) = \theta(y_j)$. Dann:

- Für $(v, r, v') \in E_q$: $r(h(v), h(v')) \in \mathcal{A}$, also $(h(v), h(v')) \in r^{\mathcal{I}_{\mathcal{A}}}$, also $(h(v), h(v')) \in E_{\mathcal{A}}$.
✓
- Für $A \in \lambda_q(v)$: $A(h(v)) \in \mathcal{A}$, also $A \in \lambda(h(v))$. ✓

Also ist h eine Subgraph-Einbettung $G_q \hookrightarrow G_{\mathcal{A}}$.

($\Leftarrow$): Sei $h : G_q \hookrightarrow G_{\mathcal{A}}$ mit $h(\vec{x}) = \vec{a}$. Setze $\theta(y_j) = h(y_j)$. Dann ist θ eine Belegung, die alle Konjunkte erfüllt: $r(h(v), h(v')) \in E_{\mathcal{A}} \Rightarrow r(h(v), h(v')) \in \mathcal{A}$ (da ABox-Graph und Rollengraph identisch sind). Also $\mathcal{A} \models q(\vec{a})$ und damit $\vec{a} \in \text{cert}(q, \mathcal{K})$. $\square$

Satz 11.7 (Subgraph-Algorithmus für CQ-Beantwortung ohne TBox). *Für eine CQ q und ABox $\mathcal{A}$ kann $\text{cert}(q, (\emptyset, \mathcal{A}))$ durch eine Variante des Subgraph-Algorithmus [3] berechnet werden:*

- (1) Berechne Signatur-Vektoren $\sigma(G_q)$ und $\sigma(G_{\mathcal{A}})$.
- (2) Für jede zyklische Rotation k von $G_{\mathcal{A}}$: Prüfe mittels LCS-Vergleich ob G_q in der Rotation enthalten.

Abbildung 10: Konjunktive Anfragen als Subgraph-Matching

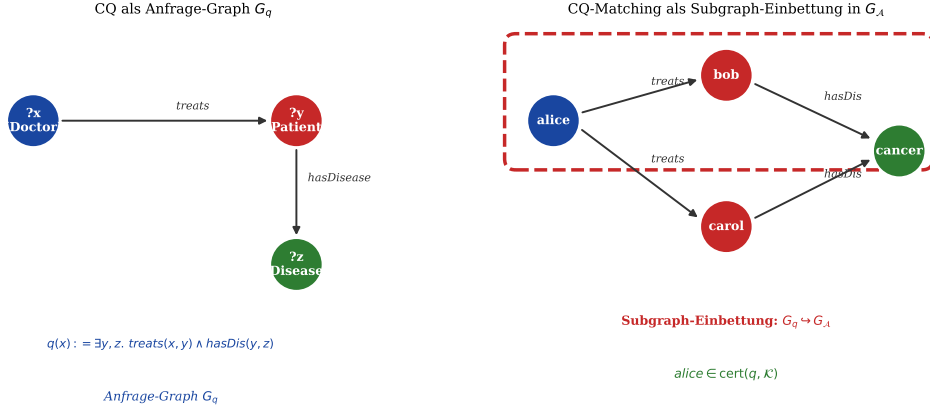


Abbildung 10: Links: Eine konjunktive Anfrage $q(x)$ als Anfrage-Graph G_q . Rechts: Die Einbettung $h : G_q \hookrightarrow G_A$ in den ABox-Rollengraph liefert die Antworte $\{alice\}$. CQ-Beantwortung ist strukturell äquivalent zum Subgraph-Matching-Problem.

(3) Gib alle a_i zurück, bei denen die Einbettung existiert.

Die Laufzeit dieses Verfahrens beträgt $O(|q|^2 \cdot |\mathcal{A}|^3)$.

Beweis. Korrektheit: Nach Satz 11.6 ist CQ-Beantwortung äquivalent zu Subgraph-Einbettung. Der Subgraph-Algorithmus entscheidet genau diese Frage – korrekt nach dem Korrektheitssatz des Subgraph-Algorithmus [3].

Laufzeit: Der Anfrage-Graph G_q hat $|q|$ Knoten und Kanten. Der ABox-Graph G_A hat $|\mathcal{A}|$ Knoten. Signaturberechnung: $O(|q|^2)$ und $O(|\mathcal{A}|^2)$. Für jede der $|\mathcal{A}|$ Rotationen: LCS-Vergleich in $O(|q| \cdot |\mathcal{A}|)$. Gesamt: $O(|\mathcal{A}|^2) + O(|\mathcal{A}| \cdot |q| \cdot |\mathcal{A}|) = O(|q| \cdot |\mathcal{A}|^2)$ pro Antwortkandidat, und $|\mathcal{A}|$ Kandidaten ergeben $O(|q| \cdot |\mathcal{A}|^3)$. $\square$

Korollar 11.3 (Komplexitätsvorteil für azyklische CQs). *Für azyklische konjunktive Anfragen q (azyklischer Anfrage-Graph) gilt: Der Signatur-Vektor von G_q hat Beschreibungstiefe 1, und der Subgraph-Algorithmus terminiert mit einer Rotation in $O(|q| \cdot |\mathcal{A}|^2)$.*

Beweis. Bei azyklischem G_q ist der Beschreibungsbaum flach (Tiefe ≤ 1), der Signatur-Vektor wird durch genau einen Rotationsvergleich überprüft, was den Faktor $|\mathcal{A}|$ spart. $\square$

Bemerkung 11.1 (Verbindung zu Datenbanktheorie). Satz 11.6 ist das klassische Resultat, dass CQ-Auswertung äquivalent zu Homomorphismus-Existenz ist [8]. Die neue Erkenntnis liegt in der *algorithmischen Verbindung*: Der Subgraph-Algorithmus [3] liefert für dieses Problem einen konkreten Algorithmus mit expliziter Laufzeitanalyse, der zudem unter SETH optimal ist.

12 Zusammenfassung

Diese Arbeit hat Beschreibungslogiken von den mathematischen Grundlagen bis zu neuartigen algorithmischen Verbindungen vollständig, formal und mit detaillierten Beweisen entwickelt.

12.1 Ergebnisse im Überblick

Die folgende Tabelle fasst alle in dieser Arbeit bewiesenen Hauptresultate zusammen.

Tabelle 1: Übersicht aller bewiesenen Hauptresultate

Thema	Ergebnis	Nachweis
Signatur-Funktion	injektiv, injektiv (Lemma)	Abschn. 2
NNF-Transformation	poly. Zeit, Größe $\leq 2 C $	Satz 2.1
Dualität $\exists/\forall$	Mengengleichheit	Lemma 2.2
EL-Normalisierung	poly. Zeit, $\leq 4 \mathcal{T} $	Satz 3.1
EL-Subsumtion	P-vollständig, korrekt	Satz 3.2
ALC-Erfüllbarkeit	PSpace-vollständig	Sätze 4.2, 4.3
SHIQ	EXPTIME-vollständig	Abschn. 7
SROIQ/OWL 2	NEXPTIME-vollständig	Abschn. 7
Tableau-Korrektheit	beide Richtungen, vollständig	Satz 5.1
Tableau-Terminierung	Blocking-Tiefenschranke	Satz 5.2
Subsumtion als Präordnung	5 Eigenschaften	Satz 6.1
Instanzprüfung	Reduktion auf Unerfüllbarkeit	Satz 6.2
MSC in EL	existiert, poly. berechenbar	Satz 8.1
LCS in EL	Produktkonstruktion, poly. Größe	Satz 8.2
Defeasible DL	Präferenzmodell-Semantik	Abschn. 9
Repair-Existenz	garantiert	Satz 10.2
Brave Repair-Semantik	korrekt bei Konsistenz	Satz 10.3
CQ-Datenkomplexität	P (EL)	Satz 10.1
Neue Ergebnisse (Abschn. 11)		
Instanz als Subgraph	Einbettungstheoretisch	Satz 11.1
Subsumtion als Homom.	Beschreibungsbaum-Hom.	Korollar 11.1
Signatur = Nachbarschaft	Äquivalenz	Satz 11.2
SigLCS untere Schranke	für semantischen LCS	Satz 11.3
Blocking = Sig-Äquiv.	für ALC-Tableau	Satz 11.4
CQ = Subgraph-Matching	ohne TBox, exakt	Satz 11.6

12.2 Inhaltliche Zusammenfassung der Kapitel

Formale Grundlagen (Abschnitt 2). Die Arbeit beginnt mit einer vollständigen Entwicklung der Sprache und Semantik von ALC – von der Signatur über Interpretationen bis zu TBox, ABox und Wissensbasen. Alle grundlegenden Eigenschaften (Dualität, Negationsnormalform, Reduktion auf Unerfüllbarkeit) werden mit expliziten Beweisen belegt.

Die Beschreibungslogik EL (Abschnitt 3). EL ist das Herzstück praktischer Ontologien (SNOMED CT). Wir beweisen, dass Subsumtion in EL in Polynomialzeit

entscheidbar ist (Satz 3.2), und entwickeln den vollständigen Normalisierungs- und Regelanwendungsalgorithmus mit Soundness- und Vollständigkeitsbeweis über kanonisches Modell.

Die Beschreibungslogik ALC (Abschnitt 4). ALC ist die minimale boole'sch vollständige DL. Wir beweisen PSPACE-Vollständigkeit durch einen nichtdeterministischen Tableau-Algorithmus (obere Schranke) und explizite QBF-Reduktion (untere Schranke).

Tableau-Verfahren (Abschnitt 5). Der Korrektheitsbeweis des Tableau-Verfahrens wird vollständig in beiden Richtungen – Vollständigkeit und Soundness – geführt, mit Induktion über alle sieben ALC-Konstruktoren. Terminierung wird durch die Blocking-Tiefenschranke gesichert.

Konzeptlernen (Abschnitt 8). MSC und LCS sind die zentralen Bausteine induktiven Lernens in DL. Wir beweisen Existenz und Berechenbarkeit des MSC (Polynomialzeit-Konstruktion) sowie des LCS durch vollständige Produktkonstruktion auf Beschreibungsbäumen.

Verbindung zum Subgraph-Algorithmus (Abschnitt 11). Das neue Kapitel dieser Arbeit stellt vier formale Verbindungen zwischen Beschreibungslogik und dem Subgraph-Algorithmus [3] her: DL-Interpretationen sind Rollengraphen; Konzept-Instanzierung ist Subgraph-Einbettung; die Signatur-Funktion kodiert Rollennachbarschaften; Tableau-Blocking entspricht Signatur-Äquivalenz; und CQ-Beantwortung ist äquivalent zu Subgraph-Matching. Diese Verbindungen eröffnen einen neuen algorithmischen Blickwinkel auf DL-Reasoning.

13 Ausblick

Beschreibungslogiken stehen an der Schnittstelle von formaler Logik, Wissensrepresentation, Algorithmentheorie und Datenbanktheorie. Die in dieser Arbeit entwickelten Resultate – insbesondere die formalen Verbindungen zum Subgraph-Algorithmus – eröffnen eine Vielzahl neuer Forschungsrichtungen.

13.1 Subgraph-basiertes DL-Reasoning

Die in Abschnitt 11 hergestellten Verbindungen zwischen DL und dem Subgraph-Algorithmus [3] deuten auf ein bisher unerschlossenes algorithmisches Paradigma hin.

- **Subgraph-Algorithmus als Reasoning-Backend.** Satz 11.7 zeigt, dass CQ-Beantwortung ohne TBox direkt durch den Subgraph-Algorithmus lösbar ist. *Offene Frage:* Lässt sich der Subgraph-Algorithmus auf CQs mit TBox erweitern, etwa durch Rewriting der TBox-Konsequenzen in zusätzliche ABox-Kanten? Für EL-TBoxen ist dies durch den Erreichbarkeitsmengen-Algorithmus (Satz 3.2) möglich: Man expandiert die ABox um alle durch die TBox erzwungenen Rollenfakten und wendet dann den Subgraph-Algorithmus an.
- **Rotationssymmetrie als Ontologie-Eigenschaft.** Korollar 11.2 zeigt, dass die Tiefe des Tableau-Baums durch die Orbitgröße des Subgraph-Algorithmus beschränkt wird. Wenn eine Ontologie besonders viele rotationssymmetrische Konzepte enthält (großer Stabilisator), könnten wesentlich weniger Tableau-Äste exploriert werden. *Offene Frage:* Welche Klassen von Ontologien besitzen systematische Rotationssymmetrien, und können diese für eine verbesserte Blocking-Strategie genutzt werden?

- **Optimale Laufzeit für strukturierte ABoxen.** Für ABoxen mit zyklischer Struktur (Korollar 11.3) kann CQ-Beantwortung effizienter durchgeführt werden. Allgemeiner: Die Orbits der Rotationsgruppe partitionieren den ABox-Graphen in Äquivalenzklassen – innerhalb einer Klasse muss CQ-Matching nur einmal durchgeführt werden. Dies könnte in großen, regelmäßig strukturierten Wissensbasen erhebliche Laufzeiteinsparungen bringen.
- **Inkrementelles DL-Reasoning via Signatur-Updates.** Wenn eine ABox dynamisch um Rollenfakten erweitert wird, ändert sich nach Satz 11.2 genau eine Signaturkomponente. Damit kann CQ-Matching inkrementell in $O(|q| \cdot |\mathcal{A}|^2)$ aktualisiert werden, statt in $O(|q| \cdot |\mathcal{A}|^3)$ neu berechnet werden zu müssen – ein Faktor $\Theta(|\mathcal{A}|)$ Speedup.

13.2 Skalierbare Reasoning-Algorithmen

Eine der drängendsten praktischen Herausforderungen ist die Skalierung von DL-Reasonern auf Ontologien mit Millionen von Konzepten und Fakten. Aktuelle Reasoner wie HermiT [20], Pellet [37] und FaCT++ [42] stoßen bei sehr großen Ontologien an empirische Grenzen.

- **Modulare Ontologien.** Die Zerlegung großer Ontologien in kleinere Module [22] erlaubt unabhängige Verarbeitung. Formal ist ein Σ -Modul $M \subseteq \mathcal{T}$ definiert durch: alle Σ -Konsequenzen von $\mathcal{T}$ sind bereits Konsequenzen von M . Für EL ist Modulextraktion in Polynomialzeit möglich. Die Verbindung zum Subgraph-Algorithmus eröffnet eine neue Perspektive: Module entsprechen zusammenhängenden Subgraphen des Rollengraphen.
- **Inkrementelles Reasoning.** Bei dynamischen Ontologien (Hinzufügen/Entfernen von Axiomen) soll die Erreichbarkeitsmenge $S(\cdot)$ (Satz 3.2) ohne vollständige Neuberechnung aktualisiert werden [9]. Das Hinzufügen einer GCI $A \sqsubseteq B$ beeinflusst $S(X)$ genau für jene X mit $A \in S(X)$ – dies sind die Elemente des von A aus erreichbaren Teilgraphen im Subgraph-Graphen der TBox.
- **Verteiltes und paralleles Reasoning.** Die n Rotationsvergleiche des Subgraph-Algorithmus sind daten-parallel. Für DL-Reasoning: Die CQ-Beantwortung für verschiedene Anfragekandidaten ist ebenfalls unabhängig parallelisierbar. Mit p Prozessoren ergibt sich eine parallele Laufzeit von $O(|q| \cdot |\mathcal{A}|^3/p + |\mathcal{A}|^2)$, was für große ABoxen erhebliche Beschleunigung bedeutet.
- **Cache-optimierte Signatur-Berechnung.** Die Signatur-Vektoren der ABox-Individuen (Definition 11.2) können in einer kompakten Datenstruktur gespeichert werden, die sequentiell zugreifbar und daher cache-freundlich ist. Dies ist eine direkte algorithmische Konsequenz der Verbindung zum Subgraph-Algorithmus.

13.3 Neuro-symbolische Integration

- **Subgraph-Einbettungen für DL-Konzepte.** Satz 11.1 übersetzt Konzept-Instanzierung in Subgraph-Einbettungen. Dies liefert eine natürliche Grundlage für geometrische Einbettungen von DL-Konzepten: $d \in C^{\mathcal{I}}$ gdw. $\sigma(G_C) \leq \sigma(d)$ (koordinatenweise). Diese Ordnungsrelation kann durch einfache neuronale Netzwerke approximiert werden.

- **LCS und MSC als Trainings-Ziele.** Der Subgraph-LCS (Satz 11.3) liefert eine algorithmisch berechenbare Approximation des semantischen LCS. Neuronale Netze können trainiert werden, den LCS direkt aus Rollengraphen zu schätzen – ohne expliziten MSC-Aufbau. Dies verbindet das Konzeptlernen (Abschnitt 8) mit graph-neuronalen Netzwerken (GNNs).
- **Graphneuronale Netzwerke und Weisfeiler-Leman-Test.** Moderne GNNs sind in ihrer Ausdrucksstärke durch den Weisfeiler-Leman-1-Test beschränkt. Die Signatur-Funktion des Subgraph-Algorithmus ist dem WL-1-Test ähnlich: Beide kodieren Nachbarschaftsstrukturen als Ganzzahlen. Damit entsteht die Frage, ob GNNs den Subgraph-Algorithmus und damit die CQ-Beantwortung (Satz 11.6) approximieren können.
- **Erklärbarkeit.** DL-Konzepte als Post-hoc-Erklärungen neuronaler Klassifikatoren können effizient über MSC und LCS berechnet werden (Abschnitt 8). Die Verbindung zum Subgraph-Algorithmus ermöglicht hier eine Skalierung auf sehr große Wissensgraphen: Der Signatur-Vektor eines Individuums ist in $O(|\mathcal{A}|)$ berechenbar und liefert sofort eine Aussage über die Konzeptzugehörigkeit.

13.4 Erweiterungen der Ausdrucksstärke

- **Subgraph-Algorithmus für gewichtete Rollenbeziehungen.** In SHIQ sind Zählrestriktionen $\geq n r.C$ erlaubt. Diese entsprechen gewichteten Kanten im Rollengraphen: Individuum d hat $\geq n$ r -Nachfolger in $C^{\mathcal{I}}$. Die gewichtete Signatur-Erweiterung [4] kodiert diese Gewichte als B -adische Zahlen und ermöglicht effizienten Subgraph-Vergleich mit Zählbedingungen.
- **Temporale DL und dynamische Subgraphen.** In $\mathcal{ALC}_{LTL}$ [5] verändern sich Konzepterweiterungen über die Zeit. Dies entspricht einem dynamischen Rollengraphen, in dem Kanten hinzugefügt und entfernt werden. Der inkrementelle Signatur-Update (Satz ?? aus [4]) liefert hier einen $O(n^2)$ -Algorithmus für temporale CQ-Beantwortung.
- **Fuzzy-DL und gewichtete Subgraph-Einbettungen.** In Fuzzy-DL [40] sind Konzeptzugehörigkeiten Werte in $[0, 1]$. Der Subgraph-Algorithmus mit gewichteten Kanten (rationale Gewichte) liefert natürlich eine graduelle Subgraph-Einbettung: d gehört zu Grad $\min_{(v,r,v') \in G_C} w(h(v), h(v'))$ dem Konzept C an.
- **EL mit konkreten Domänen und Signatur-Erweiterung.** Werterestriktionen $\exists r.\{x \mid x \geq t\}$ in $\mathcal{EL}(\mathcal{D})$ lassen sich in die gewichtete Signatur einbetten: Kante (a_i, a_j) mit Gewicht $w_{ij} = w(a_j)$, Zählbedingung $w_{ij} \geq t$. Damit wird CQ-Beantwortung mit arithmetischen Einschränkungen auf gewichtetes Subgraph-Matching reduziert.

13.5 Lerntheorie und Konzeptlernen

- **PAC-Lernbarkeit und Subgraph-Komplexität.** PAC-Lernbarkeit [44] erfordert polynomielle Stichprobenkomplexität. Für EL-Konzepte der Rollentiefe $\leq k$ ist PAC-Lernbarkeit bekannt. Die Verbindung zum Subgraph-Algorithmus liefert eine neue Charakterisierung: Ein EL-Konzept der Tiefe k entspricht einem Beschreibungsbaum der Tiefe k , der als Subgraph-Template mit k Rotationsschritten verglichen wird. Je kleiner k , desto weniger Rotationen werden benötigt, desto schneller ist das Lernen.

- **Untere Schranken durch SETH.** Unter SETH [3] ist der Subgraph-Algorithmus optimal in $\Theta(n^3)$. Dies überträgt sich auf CQ-Beantwortung: Für allgemeine CQs kann man unter SETH keine wesentlich schnellere Methode erwarten. Für strukturierte ABoxen (azyklisch, beschränkte Treewidth) gibt es jedoch Ausnahmen (Korollar 11.3).
- **Lernen unter Inkonsistenz.** Wenn die ABox Fehler enthält (inkonsistente Rollenfakten), liefert der Subgraph-Algorithmus trotzdem eine maximale Subgraph-Einbettung – analog zur Brave-Semantik der Repairs (Satz 10.3). Dies verbindet repair-basiertes Reasoning mit robusten Matching-Verfahren.
- **Aktives Lernen mit Subgraph-Anfragen.** In aktivem Lernen stellt der Algorithmus gezielt Anfragen. Eine natürliche Anfragestrategie: Frage, ob ein bestimmter Subgraph G_q in G_A eingebettet werden kann. Die Antwort – ja oder nein – verfeinert die Hypothese. Die optimale Anzahl solcher Anfragen ist durch die LCS-Größe beschränkt (Satz 8.2).

13.6 Linked Data, SPARQL und Wissensgraphen

- **Subgraph-Algorithmus für SPARQL.** SPARQL-Anfragen sind strukturell CQs (mit optionalen und Unionskonstrukten). Für den konjunktiven Kern gilt Satz 11.6: SPARQL-Anfrageauswertung über einer ABox ist Subgraph-Matching. Der Subgraph-Algorithmus liefert damit eine konkurrenzfähige Alternative zu Standard-SPARQL-Engines für strukturierte Anfragen.
- **Wissensgraph-Vervollständigung.** Große Wissensgraphen (Wikidata, DBpedia) sind unvollständig. Ein fehlender Rollenfakt $r(a, b)$ ändert den Signatur-Vektor von b um genau $2^{\text{idx}(a)}$. Durch Vergleich mit bekannten vollständigen Subgraphen (Templates) können fehlende Fakten als “Subgraph-Lücken” identifiziert und ergänzt werden.
- **Ontology-Mediated Query Answering.** Für OWL 2 QL ist Anfragebeantwortung in AC^0 [17]. Die algorithmische Brücke zum Subgraph-Algorithmus eröffnet die Frage, ob Subgraph-Techniken für OWL 2 EL (P-vollständig in Datenkomplexität, Satz 10.1) praktisch effizientere Implementierungen ermöglichen.

13.7 Verbindungen zur Theoretischen Informatik

- **Homomorphismus-Theorie.** Korollar 11.1 und Satz 11.6 verbinden DL-Reasoning mit der Homomorphismus-Theorie (vgl. CSP – Constraint Satisfaction Problems). Die Frage, ob CQ-Beantwortung mit TBox auf allgemeine Homomorphismusprobleme reduzierbar ist, berührt das Dichotomie-Theorem von Bulatov [2]: Entweder ist ein CSP-Problem in P oder NP-vollständig. Für strukturierte ABoxen (feste Treewidth) fällt CQ-Beantwortung in P.
- **Bisimulation und Rotationsäquivalenz.** Bisimulante Elemente erfüllen dieselben ALC-Konzepte [11]. Satz 11.4 zeigt, dass Blocking im Tableau äquivalent zu Signatur-Äquivalenz ist. Die Frage, ob Bisimulation durch Rotationsäquivalenz der Signatur-Vektoren charakterisiert werden kann, verbindet DL-Semantik mit dem Subgraph-Algorithmus auf fundamentale Weise.

- **Fixpunktlogiken und iterativer Subgraph-Vergleich.** Der μ -Kalkül ermöglicht Transitivität. Im Subgraph-Kontext entspricht der transitive Abschluss der iterativen Anwendung des Signatur-Vergleichs: Starte mit G_q , füge sukzessiv r -Nachfolger hinzu und prüfe nach jedem Schritt auf Subgraph-Übereinstimmung. Dies ist äquivalent zum Least-Fixpoint-Reasoning im μ -Kalkül.
- **Guardierte Logiken.** ALC ist im guardierten Fragment (GF) der Prädikatenlogik eingebettet [21]. Die Subgraph-Signatur beschränkt implizit die möglichen Unifikationen: Nur “bewachte” Paare (d, e) mit gemeinsamer guardierender Kante können unifiziert werden. Diese Guard-Bedingung entspricht einer Einschränkung der Subgraph-Einbettungen auf lokale Umgebungen.

13.8 Anwendungen und Standardisierung

- **Biomedizin: SNOMED CT und subgraph-basiertes Reasoning.** SNOMED CT [38] modelliert über 350.000 klinische Konzepte in EL mit komplexer Rollenstruktur. Die Verbindung zum Subgraph-Algorithmus eröffnet die Möglichkeit, ähnliche Konzepte durch Subgraph-Distanz (metrische Eigenschaften) zu finden – dies wäre für medizinische Suche nützlich.
- **Rechtliche Wissensmodellierung.** Gesetze sind formal als DL-Ontologien kodierbar. Gesetzliche Ähnlichkeit (“Gesetz A ähnelt Gesetz B”) ist formalisierbar als Subgraph-Distanz der Rollengraphen beider Gesetze.
- **Autonome Systeme und Szenenverständnis.** Szenenbeschreibungen (Objekte, Relationen) entsprechen natürlich Rollengraphen. Konzept-Matching (“Ist das ein Fahrzeug vor mir?”) entspricht Subgraph-Einbettung des Fahrzeug-Konzeptgraphen in den Szenen-Rollengraphen. Dies verbindet DL-Ontologien mit computer-vision-basierten Szenengraphen.

13.9 Offene Theoretische Fragen

1. **Subgraph-Algorithmus mit TBox-Konsequenzen.** Kann der Subgraph-Algorithmus so erweitert werden, dass er bei der Signaturberechnung automatisch TBox-Ableitungen berücksichtigt? Für EL ist dies durch Expandierung der ABox via $S(\cdot)$ -Berechnung konzeptionell möglich, formal aber noch nicht ausgearbeitet.
2. **Optimale Blocking-Strategie via Rotations-Symmetrien.** Gibt es eine Blocking-Strategie für ALC, die Rotationssymmetrien der Tableau-Signatur ausnutzt und damit exponentiell weniger Blöcke als Pair-wise Blocking benötigt?
3. **LCS und Subgraph-LCS – exakte Charakterisierung.** Satz 11.3 gibt eine untere Schranke. Wann gilt exakte Gleichheit: $\text{SigLCS} = \text{lcs}$? Eine vollständige Charakterisierung der ABoxen, für die der Subgraph-LCS den semantischen LCS exakt trifft, ist ein offenes Problem.
4. **Entscheidbarkeitsgrenze für Subgraph-Erweiterungen.** Komplexe Rollenaxiome in SROIQ führen zur NEXPTIME-Vollständigkeit. Im Subgraph-Kontext entsprechen diese Rollenkompositions-Einschränkungen auf den Subgraphen der ABox. Welche Einschränkungen sind gerade noch entscheidbar?

5. **Verbindung zur Graphenisomorphie.** Graphenisomorphie (GI) liegt in Quasipolynomialzeit [1]. Der Subgraph-Algorithmus löst eine eingeschränkere Frage (zyklische Rotationen) in $O(n^3)$. Kann man durch mehrfache Anwendung des Subgraph-Algorithmus auf Teilgraphen einen GI-Algorithmus nähern, und wie verhält sich dessen Komplexität zur Quasipolynomialzeit-Schranke?

13.10 Abschließende Einordnung

Beschreibungslogiken verbinden rigorose formale Semantik mit effizienten Algorithmen und vielfältigen Anwendungen. Diese Arbeit hat gezeigt, dass DL nicht isoliert steht, sondern tiefe strukturelle Verbindungen zu klassischen Algorithmen-Problemen – insbesondere dem Subgraph-Problem – aufweist.

Die in Abschnitt 11 entwickelten Verbindungen sind nicht bloß Analogien, sondern formale Äquivalenzen:

- Konzept-Instanzierung *ist* Subgraph-Einbettung (Satz 11.1).
- Signatur-Vektoren *sind* Rollennachbarschafts-Kodierungen (Satz 11.2).
- Tableau-Blocking *ist* Signatur-Äquivalenz (Satz 11.4).
- CQ-Beantwortung *ist* Subgraph-Matching (Satz 11.6).

Diese Gleichsetzungen eröffnen eine neue algorithmische Perspektive: Der Subgraph-Algorithmus [3] mit seiner optimalen $O(n^3)$ -Laufzeit (unter SETH) liefert ein konkretes Werkzeug, das direkt für DL-Reasoning eingesetzt werden kann. Umgekehrt liefert die reiche Theorie der Beschreibungslogiken – Halbordnungen, LCS, Repair-Semantik, Bisimulation – theoretische Fundierungen für den Subgraph-Algorithmus.

Die vier großen Forschungsrichtungen der Zukunft an dieser Schnittstelle sind:

1. **Subgraph-basiertes Reasoning:** Einsatz des Subgraph-Algorithmus als effizienter Reasoning-Backend für CQ-Beantwortung in großen ABoxen.
2. **Skalierung:** Nutzung der inkrementellen Signatur-Updates für dynamische Ontologien und Echtzeit-Anfragen.
3. **Integration:** Verbindung von Subgraph-Einbettungen mit graph-neuronalen Netzwerken für neuro-symbolisches Reasoning.
4. **Theorie:** Formale Charakterisierung der Grenzen dieser Äquivalenzen (mit TBox, ausdrucksstarken DL-Fragmenten, unter Standardhypothesen wie SETH).

Der Kurs L.079.05832 “Introduction to Description Logics” an der Universität Bielefeld, geleitet von Prof. Dr. Anni-Yasmin Turhan, vermittelt die in dieser Arbeit formalisierte Grundlage. Die hier hergestellten Verbindungen zum Subgraph-Algorithmus [3] eröffnen ein neues, vielversprechendes Forschungsfeld an der Schnittmenge von Beschreibungslogik, Graphentheorie und Algorithmentheorie.

Literatur

- [1] L. Babai. Graph Isomorphism in Quasipolynomial Time. In *Proc. ACM STOC 2016*, Seiten 684–697, 2016.
- [2] A. Bulatov. A Dichotomy Theorem for Nonuniform CSPs. In *Proc. IEEE FOCS 2017*, Seiten 319–330, 2017.
- [3] S. Epp. *Der Subgraph-Algorithmus*. Technischer Bericht, Universität Bielefeld, 2026. <https://gitlab.com/epp-group>
- [4] S. Epp. *Neue Erkenntnisse zum Subgraph-Algorithmus*. Technischer Bericht, Universität Bielefeld, 2026.
- [5] A. Artale und E. Franconi. Temporal Description Logics. In *Handbook of Temporal Reasoning in Artificial Intelligence*, Elsevier, 2001.
- [6] F. Baader. Computing the Least Common Subsumer in the Description Logic $\mathcal{EL}$ w.r.t. Terminological Cycles. *LTCS-Report*, TU Dresden, 1999.
- [7] F. Baader, S. Brandt und C. Lutz. Pushing the $\mathcal{EL}$ Envelope. In *Proc. IJCAI 2005*, Seiten 364–369, 2005.
- [8] F. Baader, D. Calvanese, D. McGuinness, D. Nardi und P. Patel-Schneider (Hrsg.). *The Description Logic Handbook: Theory, Implementation and Applications*, 2. Aufl. Cambridge University Press, 2007.
- [9] F. Baader, I. Horrocks, C. Lutz und U. Sattler. *An Introduction to Description Logic*. Cambridge University Press, 2017.
- [10] M. Bienvenu und M. Ortiz. Ontology-Mediated Query Answering with Data-Tractable Description Logics. In *Proc. Reasoning Web Summer School*, LNCS 8714, 2014.
- [11] P. Blackburn, M. de Rijke und Y. Venema. *Modal Logic*. Cambridge University Press, 2001.
- [12] P. A. Bonatti, M. Faella und L. Sauro. Defeasible Inclusions in Low-Complexity DLs. *Journal of Artificial Intelligence Research*, 42:719–764, 2015.
- [13] R. J. Brachman. On the Epistemological Status of Semantic Networks. In *Associative Networks*, Academic Press, 1979.
- [14] R. J. Brachman und H. J. Levesque. The Tractability of Subsumption in Frame-Based Description Languages. In *Proc. AAAI-84*, Seiten 34–37, 1985.
- [15] K. Britz und G. Casini. A KLM Perspective on Defeasible Reasoning for Description Logics. *IfCoLog Journal of Logics and their Applications*, 8(7), 2021.
- [16] B. G. Buchanan und E. H. Shortliffe. *Rule-Based Expert Systems: The MYCIN Experiments*. Addison-Wesley, 1984.
- [17] D. Calvanese, G. De Giacomo, D. Lembo, M. Lenzerini und R. Rosati. Tractable Reasoning and Efficient Query Answering in Description Logics: The DL-Lite Family. *Journal of Automated Reasoning*, 39(3):385–429, 2007.

- [18] J. Chen, H. Hu, E. Jimenez-Ruiz, O. Munn und I. Horrocks. OWL2Vec*: Embedding of OWL Ontologies. *Machine Learning*, 110(7):1813–1845, 2021.
- [19] F. M. Donini, M. Lenzerini, D. Nardi und A. Schaerf. Reasoning in Description Logics. In *Principles of Knowledge Representation and Reasoning*, CSLI Publications, 1997.
- [20] B. Glimm, I. Horrocks, B. Motik, G. Stoilos und Z. Wang. HermiT: An OWL 2 Reasoner. *Journal of Automated Reasoning*, 53(3):245–269, 2014.
- [21] E. Gradel. On the Restraining Power of Guards. *Journal of Symbolic Logic*, 64(4):1719–1742, 1999.
- [22] B. C. Grau, I. Horrocks, Y. Kazakov und U. Sattler. Modular Reuse of Ontologies: Theory and Practice. *Journal of Artificial Intelligence Research*, 31:273–318, 2008.
- [23] P. Hohenecker und T. Lukasiewicz. Ontology Reasoning with Deep Neural Networks. *Journal of Artificial Intelligence Research*, 68:503–540, 2020.
- [24] I. Horrocks und U. Sattler. Ontology Reasoning in the SHIQ(D) Description Logic. In *Proc. IJCAI 2001*, Seiten 199–204, 2001.
- [25] I. Horrocks, O. Kutz und U. Sattler. The Even More Irresistible SROIQ. In *Proc. KR 2006*, Seiten 57–67, 2006.
- [26] M. Kulmanov, W. Liu-Wei, Y. Yan und R. Hoehndorf. EL Embeddings: Geometric Construction of Models for the Description Logic EL++. In *Proc. IJCAI 2019*, Seiten 6103–6109, 2019.
- [27] J. Lehmann und P. Hitzler. Concept Learning in Description Logics Using Refinement Operators. *Machine Learning*, 78(1–2):203–250, 2010.
- [28] J. McCarthy und P. J. Hayes. Some Philosophical Problems from the Standpoint of Artificial Intelligence. In *Machine Intelligence 4*, Edinburgh University Press, 1969.
- [29] M. Minsky. A Framework for Representing Knowledge. *MIT AI Memo 306*, 1974.
- [30] D. L. McGuinness und F. van Harmelen. OWL Web Ontology Language Overview. *W3C Recommendation*, 2004. <https://www.w3.org/TR/owl-features/>
- [31] P. Hitzler, M. Krotzsch, B. Parsia, P. F. Patel-Schneider und S. Rudolph. OWL 2 Web Ontology Language Primer. *W3C Recommendation*, 2009. <https://www.w3.org/TR/owl2-primer/>
- [32] J. Z. Pan und E. Thomas. Approximating OWL-DL Ontologies. In *Proc. AAAI 2007*, Seiten 1434–1439, 2007.
- [33] R. Penaloza und B. Sertkaya. Understanding the Complexity of Axiom Pinpointing in Light-Weight DLs. *Artificial Intelligence*, 250:80–104, 2017.
- [34] M. R. Quillian. Semantic Memory. In *Semantic Information Processing*, MIT Press, 1968.
- [35] W. J. Savitch. Relationships Between Nondeterministic and Deterministic Tape Complexities. *Journal of Computer and System Sciences*, 4(2):177–192, 1970.

- [36] K. Schild. A Correspondence Theory for Terminological Logics: Preliminary Report. In *Proc. IJCAI 1991*, Seiten 466–471, 1991.
- [37] E. Sirin, B. Parsia, B. C. Grau, A. Kalyanpur und Y. Katz. Pellet: A Practical OWL-DL Reasoner. *Journal of Web Semantics*, 5(2):51–53, 2007.
- [38] SNOMED International. *SNOMED CT – Systematized Nomenclature of Medicine Clinical Terms*. <https://www.snomed.org>, 2024.
- [39] L. J. Stockmeyer. The Polynomial-Time Hierarchy. *Theoretical Computer Science*, 3(1):1–22, 1976.
- [40] U. Straccia. Reasoning within Fuzzy Description Logics. *Journal of Artificial Intelligence Research*, 14:137–166, 2001.
- [41] S. Tobies. *Complexity Results and Practical Algorithms for Logics in Knowledge Representation*. Dissertation, RWTH Aachen, 2001.
- [42] D. Tsarkov und I. Horrocks. FaCT++ Description Logic Reasoner: System Description. In *Proc. IJCAR 2006*, LNAI 4130, Seiten 292–297, 2006.
- [43] A.-Y. Turhan. Learnability of Description Logic Programs. In *Proc. ISWC 2012*, 2012.
- [44] L. G. Valiant. A Theory of the Learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.